

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Panchenko, Dmitry. "Multiple Linear Regression." Statistics for Applications, Fall 2006, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-443-statistics-for-applications-fall-2006/dbc17cbfdac61daaff75056992d0af9a_section15.pdf

- **Required pages:** Entire Lecture
- **Estimated reading time:** 2 hours

Lesson 1 - Linear Regression Analysis

December 8, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 1

1 LINEAR REGRESSION ANALYSIS

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Undergraduate-Level Statistics
Keywords	Endogenous Variable (Dependent Variable), Exogenous Variable (Independent Variable), Scatter Plot, Outlier, Leverage Point, Ordinary Least Square Linear Regression (OLS), Parameter (Coefficient), R2, Adjusted R2, Cook's Distance (Cook's D), Influential Point, Influence Plot

In this course, we will use econometrics to analyze how financial data behaves and interacts. Econometrics uses mathematics and statistical inference to explore the relationship among different financial variables. There are several techniques and models in econometrics. In this lesson, we will start by applying linear regression analysis to demonstrate some basic concepts in econometrics. Linear regression analysis is the most frequently used analytical technique in finance and is the foundation of many more advanced econometrics theories. This lesson will also cover the basic steps of implementing linear regression analysis. These basic steps are applicable to future advanced econometrics analyses. Readers are expected to have prior knowledge of undergraduate statistics.

```
[1]: # Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf

plt.rcParams["figure.figsize"] = (12, 9)
```

1.1 1. Prepare a Project for Linear Regression Analysis

Assume you work for an asset management company as an equity analyst. Your manager asks you to find out what caused the recent increase in Coca Cola's stock (NYSE: KO) price and if there is still potential for the price to keep growing. In this situation, what would you do? Here are three steps to think about:

1.1.1 1.1 Identify Potential Factors Influencing KO's Stock Price

You can do some research to identify potential factors that impact the stock price. It could be the company's sales. It could be that the whole stock market is going up. There might be other factors that explain KO's stock price increase based on your research and understanding of the company. Once you identify the factors, you can start to collect data for analysis.

1.1.2 1.2. Quantify the Relationship between KO's Stock Price and Selected Factors

Once the factors are determined and the data is collected, we need a system in place to tell us how each factor influences the stock price. For example, we would like to know not only if the fluctuation in sales will increase/decrease the stock price but also how much the stock price will move when the sales numbers move. We need to build a model to provide this information. In this lesson, we will use linear regression analysis to build this model. Then, we will assess how accurate the model is and adjust the model until we get a satisfactory one.

1.1.3 1.3 Forecast the future stock price

After we get the regression model, we can create several scenarios to predict the stock price move. For example, if KO's revenue is one of the factors in the model, we can simulate how much the stock price would change given a 0.5% increase, 1% increase, and 2% increase in revenue. This exercise will help us predict KO's stock price increase potential.

1.2 2. Endogenous Variable vs. Exogenous Variable

Assume you have identified that the following three factors are most impactful on KO's stock price: KO's revenue, Pepsi's stock price, and the Dow Jones Industrial Average Index. We will use these three factors to create a linear regression model to predict KO's next-day closing stock price. Finance theory tells us a stock excess return is a better variable to analyze than the actual price. The excess return is the actual return minus risk-free interest rate (usually Treasury Bill Rates). Therefore, we will convert all our variables into returns or ratios. In our modeling dataset, we will have the following variables:

KO's stock excess return (%) KO's sales growth (%) DOW 30 Index excess return (%)
Pepsi's stock excess return (%)

In order to use comparable variables to conduct modeling work, all variables are quarterly ratios (quarterly excess returns and quarterly sales growth).

Once we have our dataset ready, we need to decide which variables are endogenous variables and which are exogenous variables. An **endogenous variable**, also called a **dependent variable**, is a variable whose value is determined by the model. An **exogenous variable**, also called an **independent variable**, is a variable whose value is determined outside of the model or is given to us. In our case, we want to build a regression model to understand and predict how KO's stock

excess return will move when the three factors change. *KO*'s stock excess return is an endogenous variable. The other three variables are exogenous variables because their value is not decided by the model.

Figure 1: Relationship between Exogenous Variables and Endogenous Variable

1.3 3. Check Relationship between Endogenous Variable and Exogenous Variables

After defining endogenous variables and exogenous variables for our regression model, we will use some graphical tools to investigate the relationships of our selected variables. This step is important because these graphs can give us information as to how to structure our regression model and decide if some selected variables and data points should be excluded. The graphic tool we will use is a **scatter plot**. A scatter plot is a two-dimensional graph with one variable on one axis and another variable on another axis. A scatter plot can show us the relationship of two variables. So what exactly is a scatter plot used for?

First, a scatter plot can check for the linear relationship between the endogenous variable and the exogenous variables. One of the assumptions in running regression analyses is that the endogenous variable has a linear relationship with exogenous variables. By looking at the scatter plot, we can determine if we can draw a straight line where most of the points will be around the line.

Second, a scatter plot can be used to check for independence between the endogenous variable and exogenous variables. Continuing from the previous paragraph, if the straight line drawn is either a horizontal line or a vertical line, it means no matter how much the value of one variable moves, the value of the other variable always stays within a tight range. In this case, the two variables are independent of each other. If an exogenous variable is independent of the endogenous variable, we should drop this exogenous variable from the model.

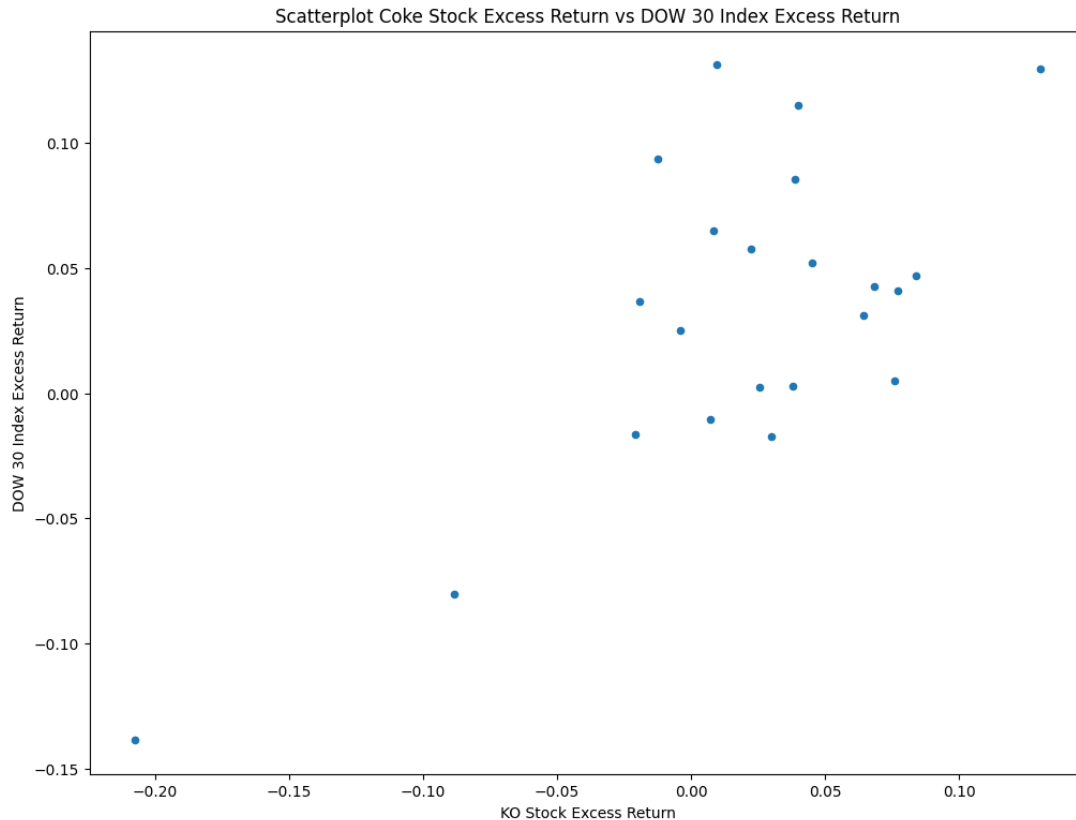
Lastly, a scatter plot can be used to find extreme points that are well-separated from the other points. There are two types of extreme points. One is called an outlier. An **outlier** is a point that doesn't follow the general trend of the other points. The other one is called a leverage point. A **leverage point** has extreme value for exogenous variables. Both types may or may not impact the estimated result of the regression model. Try to run one model with them and another model without them. Then, compare the two model results to see if these extreme points are crucial to the final model result.

Figure 2: Scatter Plot—KO Stock Excess Return vs. Dow 30 Index Excess Return

```
[2]: # Download the dataset
data_set = pd.read_csv("fin_data_set.csv")

[3]: # Create scatter plot
scatt = data_set.plot.scatter(
    x="Coke_Q_EX_R",
    y="Dow_Q_EX_R",
    title="Scatterplot Coke Stock Excess Return vs DOW 30 Index Excess Return",
)
```

```
scatt.set(xlabel="KO Stock Excess Return", ylabel="DOW 30 Index Excess Return")
plt.show()
```



The scatter plot in Figure 2 shows that the *KO* stock excess return and *Dow 30* Index excess return tend to move in the same direction within our dataset. A straight line can be drawn from the lower left to the upper right. There are two extreme points in the plot, but they are positioned along the straight line.

We can also use the **Pearson Correlation** metric to check the linear relationship of two variables. The value of the Pearson Correlation is between -1 and 1 . If the value of two variables tends to move in the same direction, the correlation is positive. If the two variables tend to move in opposite directions, the number is negative. A correlation of 1 means the two variables have a perfectly positive linear relationship.

Figure 3: Histogram, Correlation, and Scatter Plot Graph Matrix For Independent Variables and Dependent Variable

```
[4]: # Function to calculate correlation coefficient between two arrays

def corr(x, y, **kwargs):
```

```

# Calculate the value
coef = np.corrcoef(x, y)[0][1]
# Make the label
label = r"$\rho$ = " + str(round(coef, 2))

# Add the label to the plot
ax = plt.gca()
ax.annotate(label, xy=(0.3, 0.15), size=20, xycoords=ax.transAxes)

```

```

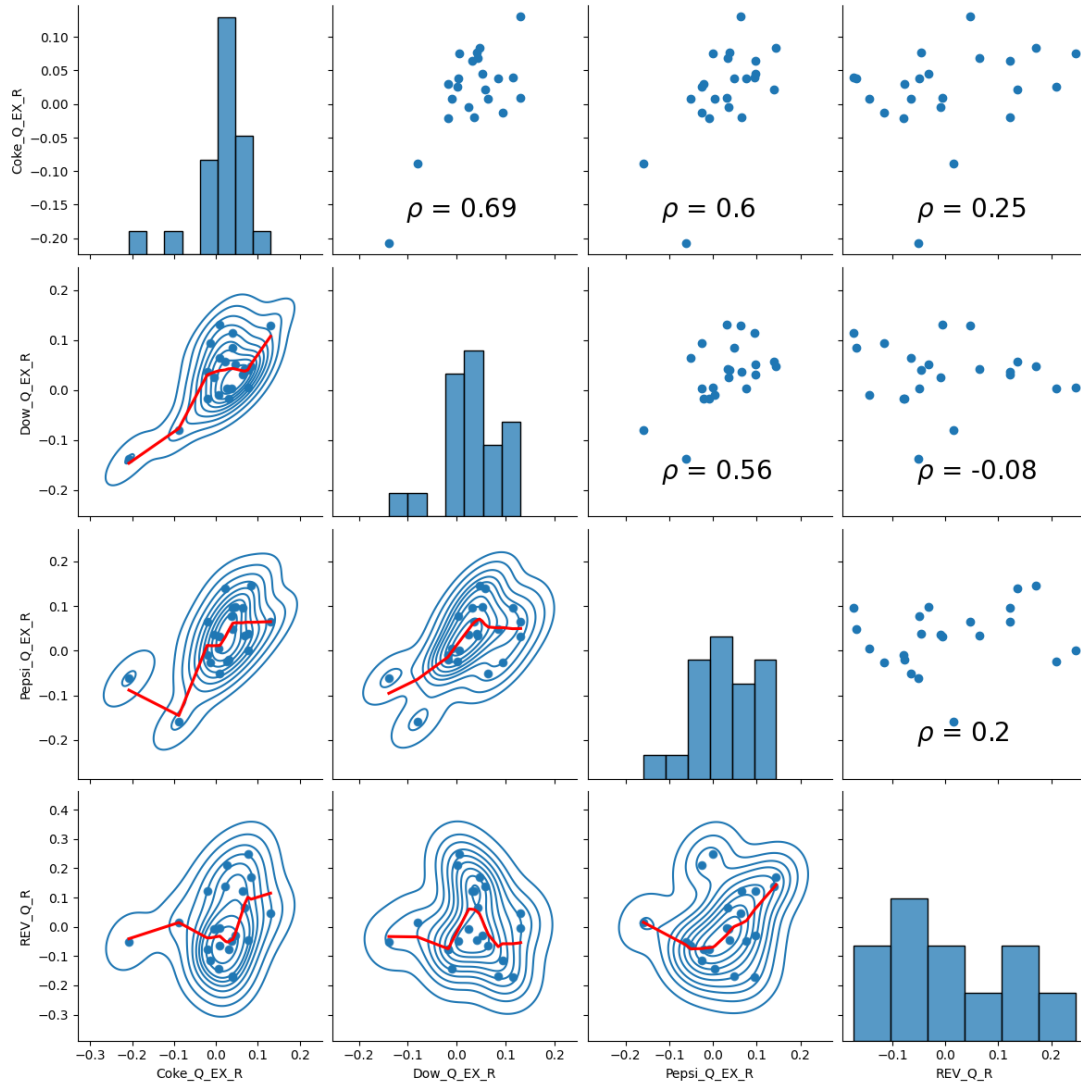
[5]: # Create the default pairplot
grid = sns.pairplot(
    data_set, vars=["Coke_Q_EX_R", "Dow_Q_EX_R", "Pepsi_Q_EX_R", "REV_Q_R"],
    height=3
)

# Map a scatter plot and Pearson correlation coefficient to the upper triangle
grid = grid.map_upper(plt.scatter)
grid = grid.map_upper(corr)

# Map a histogram to the diagonal
# grid = grid.map_diag(plt.hist)

# Map a density plot and regression line to the lower triangle
grid = grid.map_lower(sns.kdeplot)
grid = grid.map_lower(sns.regplot, lowess=True, line_kws={"color": "red"})

```



Data Period: 2016Q2 – 2021Q3 Data Source: Yahoo Finance and Coca Cola’s website
Coke_Q_EX_R: KO Quarterly Stock Excess Return *Dow_Q_EX_R*: DOW 30 Index Quarterly Excess Return *Pepsi_Q_EX_R*: Pepsi Quarterly Stock Excess Return
REV_Q_R: KO Quarterly Revenue change %

Figure 3 provides a summary of the relationships of variables in *KO*’s stock excess return analysis. The matrix is split into three parts.

- The lower left triangle of the matrix shows the scatter plots for all two-way combinations of the four variables.
- The graphs of the diagonal of the matrix are the histograms of the four variables.
- The upper right triangle of the matrix provides the correlation values of all two-way combinations of the four variables.

The scatter plot from the *KO* return and *Dow* return shows the two variables have a good positive

linear relationship, as is the case for the *KO* return and Pepsi return. In both scatter plots, the data points also form an oval shape, more or less. The red line is also tilted upward. These signs suggest these two variables are good choices for modeling *KO*'s return. In the scatter plot for *KO*'s return and *KO*'s revenue growth, we don't see as clear an oval shape, and the red line is not as straight and sloped upward as the other two graphs. Their correlation values on the upper right also confirm the information from the graphs. Dow return and Pepsi return have good positive correlation values with *KO*'s return. However, *KO*'s revenue growth and return have a smaller correlation.

1.4 4. Estimate the Regression Model

Now, let's start to build the regression model. The method we are going to use here is ordinary least square (OLS) linear regression. There are two types of OLS linear regression models: **simple OLS** and **multiple OLS**. If there is only one exogenous variable vs. one endogenous variable, then it is called simple OLS linear regression. If there is more than one exogenous variable vs. one endogenous variable, then it is called multiple OLS linear regression. Since we have three exogenous variables for our model, we will build a multiple OLS linear regression.

Here is the linear regression model we are going to estimate:

$$\text{KO excess return} = b_0 + b_1 \text{Dow excess return} + b_2 \text{Pepsi excess return} + b_3 \text{KO revenue growth}$$

b_0 , b_1 , b_2 and b_3 are called **parameters** or **coefficients**. We will use the OLS method to estimate the values of parameters in the model. Once the values of the parameters are obtained from the OLS calculation, we call these values **estimates**.

Figure 4: Linear Regression Result for KO Excess Return Model

[6]: `# Linear regression result`

```
result = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + Pepsi_Q_EX_R + REV_Q_R", data=data_set
).fit()
result.summary()
```

[6]:

Dep. Variable:	Coke_Q_EX_R	R-squared:	0.604			
Model:	OLS	Adj. R-squared:	0.538			
Method:	Least Squares	F-statistic:	9.150			
Date:	Mon, 24 Nov 2025	Prob (F-statistic):	0.000677			
Time:	06:51:52	Log-Likelihood:	38.570			
No. Observations:	22	AIC:	-69.14			
Df Residuals:	18	BIC:	-64.78			
Df Model:	3					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.0080	0.011	-0.716	0.483	-0.032	0.015
Dow_Q_EX_R	0.6197	0.195	3.173	0.005	0.209	1.030
Pepsi_Q_EX_R	0.2187	0.179	1.221	0.238	-0.158	0.595
REV_Q_R	0.1435	0.088	1.632	0.120	-0.041	0.328

Omnibus:	1.644	Durbin-Watson:	2.307
Prob(Omnibus):	0.439	Jarque-Bera (JB):	1.411
Skew:	-0.564	Prob(JB):	0.494
Kurtosis:	2.481	Cond. No.	24.1

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: # Parameters with 6 significant digits
result.summary2().tables[1]
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Intercept	-0.008017	0.011189	-0.716476	0.482890	-0.031524	0.015490
Dow_Q_EX_R	0.619723	0.195287	3.173393	0.005261	0.209440	1.030006
Pepsi_Q_EX_R	0.218664	0.179107	1.220854	0.237893	-0.157627	0.594955
REV_Q_R	0.143481	0.087912	1.632093	0.120031	-0.041216	0.328178

Figure 4 shows the result of the regression. Let's look at the estimates of the three variables. What does the estimate mean? The estimate for Dow excess return is 0.619723. It means when Dow excess return increases by 0.01 (1%), KO excess return will increase by 0.00619723 (0.619723%).

1.5 5. Review the Model

Our next question will focus on how good our estimated model is. There are several ways to assess and test the fitness of the model. We will first look at the performance of each exogenous variable in the model and then assess the overall model fit.

1.5.1 5.1 Performance of Each Exogenous Variable

Under the section of coefficient in the result table, there is a column called $Pr(>|t|)$. This column shows p -values for each coefficient. A p -value is a metric to indicate whether the exogenous variable can explain the change of the endogenous variable. The lower the number of the p -value, the better the explanatory ability this exogenous variable has regarding the endogenous variable. In many research projects, 0.05 is the level that determines whether an exogenous variable is significant. From our result, the only exogenous variable that is significant is Dow excess return (p -value: $0.00526 < 0.05$). Does that mean we can just drop the other two variables from the model? There are at least two points we can consider before we make that decision.

1. Does KO's excess return and KO's revenue growth exhibit a linear relationship?

From the scatter plot for both variables, we know they have a weak linear relationship. Can we transform KO's revenue growth into a different data series and then check the relationship again? We can try to take the square root or the logarithm of KO's revenue growth and draw the scatter plot again.

2. Are exogenous variables also correlated to each other?

The correlation between Dow excess returns and Pepsi excess returns is 0.56 based on the scatter plot matrix. The two variables are medium positively correlated. When there are two or more correlated exogenous variables in a regression model, oftentimes, the exogenous variable with a higher correlation to the endogenous variable will absorb all the explanatory ability from other

correlated exogenous variables, making other correlated exogenous variables not significant in the model.

The above two points help us rethink our choice of exogenous variables in the model and the way to handle them. We will discuss these two points in more detail and offer possible solutions in the following lessons.

1.5.2 5.2 R^2 and Adjusted R^2

Now, let's look at the general model fit. R^2 , the coefficient of determination, is a commonly used metric to assess the model fit. R^2 tells us how much of the variation in the endogenous variable can be explained by the variation of all exogenous variables in the model. The value of R^2 is between 0 and 1. If R^2 is 1, it means the model perfectly explains the *KO* excess return behavior. In other words, given a set of numbers from all exogenous variables, the model can 100% correctly produce the number for the endogenous variable. In general, the higher the value of R^2 , the better the model fit.

In our model, the R^2 is 0.604. It means that the change in Dow excess return, Pepsi excess return, and *KO* revenue growth together can explain 60% of the change in *KO* excess return.

If your manager thinks 0.604 is not satisfactory, you may be asked to improve the model. Usually, adding a new exogenous variable is an easy starting point. Adding more exogenous variables to a model usually improves the model's R^2 . However, it can cause the issue of a model overfitting. What is model overfitting? It means the model can do an excellent job predicting the endogenous variable from the data sample that is used to build the model. However, when you put a different dataset with the endogenous variable and exogenous variables into the model, the model will give a poor forecast of the endogenous variable with the new values for the exogenous variables.

An **adjusted R^2** can address this issue. What does an adjusted R^2 intend to adjust? Although adding new independent variables to a regression model will never lower R^2 , it will add complexity to the model. It can also cause the model overfitting issue as described above. Hence, we have to ask if the extra complexity added to the model is worth it. This is what an adjusted R^2 does. Like R^2 , an adjusted R^2 is between 0 and 1. An adjusted R^2 is always lower than R^2 . An adjusted R^2 will increase if the degree of improvement from adding one exogenous variable is greater than the penalty imposed by one additional exogenous variable to the model.

In our model, the adjusted R^2 is 0.538. It is hard to tell if this is a good adjusted R^2 . In practice, a researcher usually builds several versions of the regression models and then compares their adjusted R^2 to pick a final model. Model building and tweaking is an iterative process; it is rarely the case that the first model is the best model for any project.

1.6 6. Outliers and Influence

In the previous section, we reviewed the performance of the exogenous variables in the model and of the model as a whole. In this section, we will look into the impact of each data point from the data sample used to build the regression model.

In the scatter plot section, we discussed extreme values in the dataset. We introduced outliers and leverage points. Not all of them will influence the result of the regression model. The following visual examples demonstrate how an extreme data point can impact the regression result. The red

dot is the extreme point. The black line is the regression line without the extreme point. The red line is the regression line with the extreme point.

Figure 5: The Impact of an Outlier on the Regression Model Result

```
[8]: # Generate 30 random points
np.random.seed(42)
n = 30
x = np.random.uniform(0, 5, n)
y = 1 + 3 * x + np.random.normal(0, 1, n)

# Add outlier at (2.5, 17)
x_with_out = np.append(x, 2.5)
y_with_out = np.append(y, 17)

[9]: # Plot best line of fit for 30 original points only (in black) and for
      ↪ 30+outlier (in red)
plt.title("Outlier", {"size": 14})
plt.plot(x, y, "o") # scatterplot
plt.plot(2.5, 17, "ro") # Outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(ao, bo) = np.polyfit(x_with_out, y_with_out, 1)
plt.plot(
    x_with_out,
    ao * x_with_out + bo,
    "r-",
    label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()
```

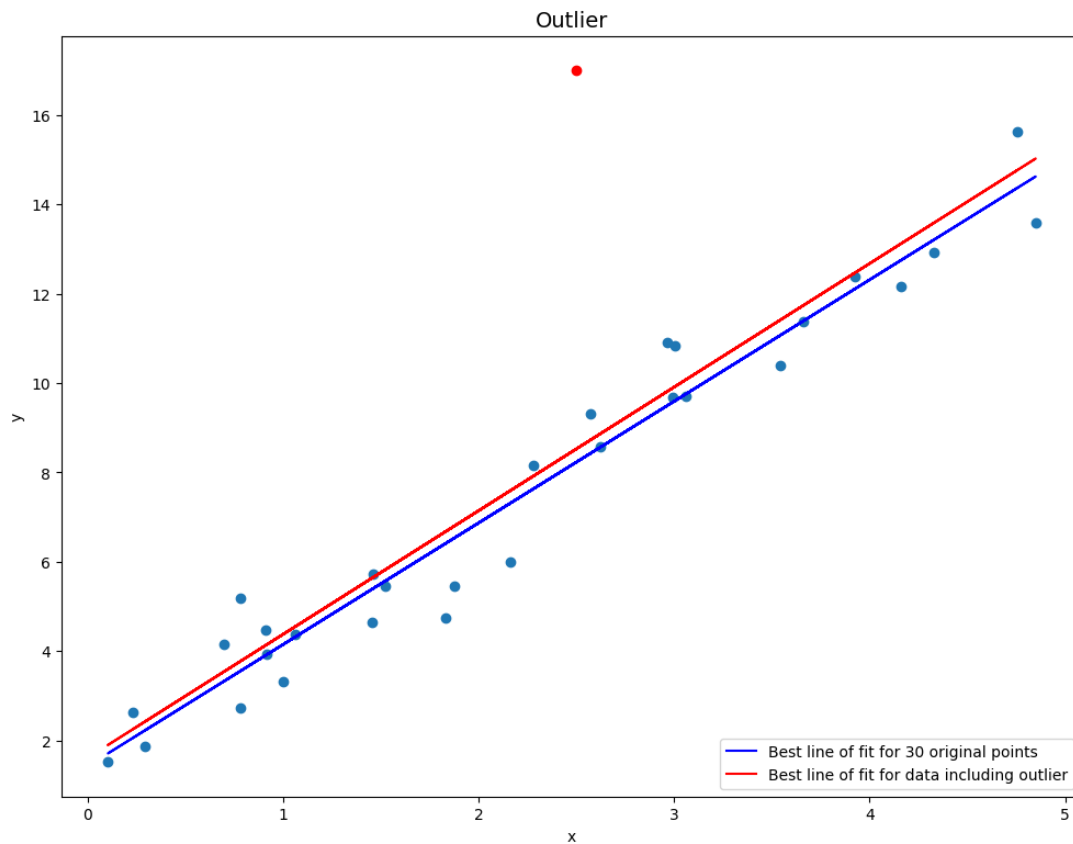


Figure 6: The Impact of a Non-Influential Leverage Point on the Regression Model Result

```
[10]: # Add non-influential leverage point at (8, 25)
x_with_lev = np.append(x, 8)
y_with_lev = np.append(y, 25)

[11]: # Plot best line of fit for 30 original points only (in black) and for 30+lev_
      ↪ (in red)
plt.title("Leverage (non influential)", {"size": 14})
plt.plot(x, y, "o") # scatterplot
plt.plot(8, 25, "ro") # non-influential outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(al, bl) = np.polyfit(x_with_lev, y_with_lev, 1)
plt.plot(
```

```

x_with_lev,
a1 * x_with_lev + b1,
"r-",
label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()

```

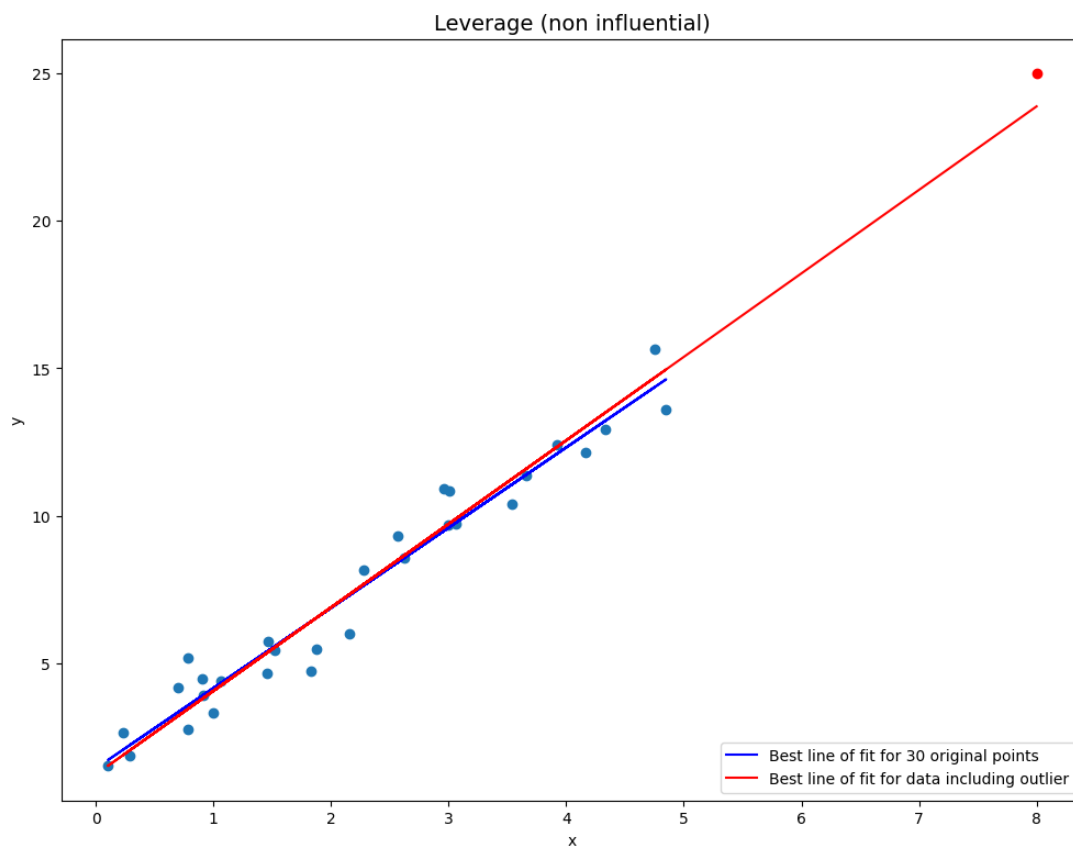


Figure 7: The Impact of an Influential Leverage Point on the Regression Model Result

```

[12]: # Add non-influential leverage point at (8, 25)
x_with_inf = np.append(x, 8)
y_with_inf = np.append(y, 25)

[13]: # Plot best line of fit for 30 original points only (in black) and for 30+lev_
      ↪ (in red)
plt.title("Leverage (influential)", {"size": 14})

```

```

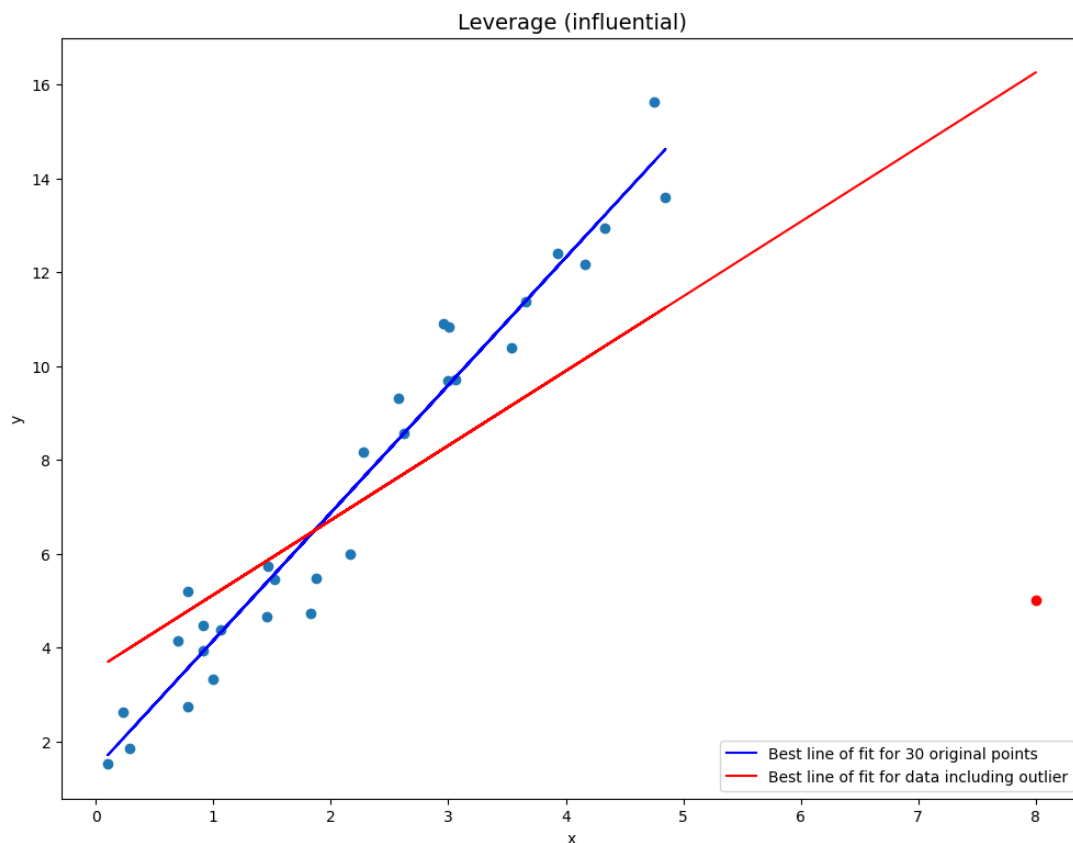
plt.plot(x, y, "o") # scatterplot
plt.plot(8, 5, "ro") # influential outlier in red

# line of fit for 30 original points only
(a, b) = np.polyfit(x, y, 1)
plt.plot(x, a * x + b, "b-", label="Best line of fit for 30 original points")

# line of fit for 30+outlier
(ai, bi) = np.polyfit(x_with_inf, y_with_inf, 1)
plt.plot(
    x_with_inf,
    ai * x_with_inf + bi,
    "r-",
    label="Best line of fit for data including outlier",
)

plt.xlabel("x")
plt.ylabel("y")
plt.legend(loc="lower right")
plt.show()

```

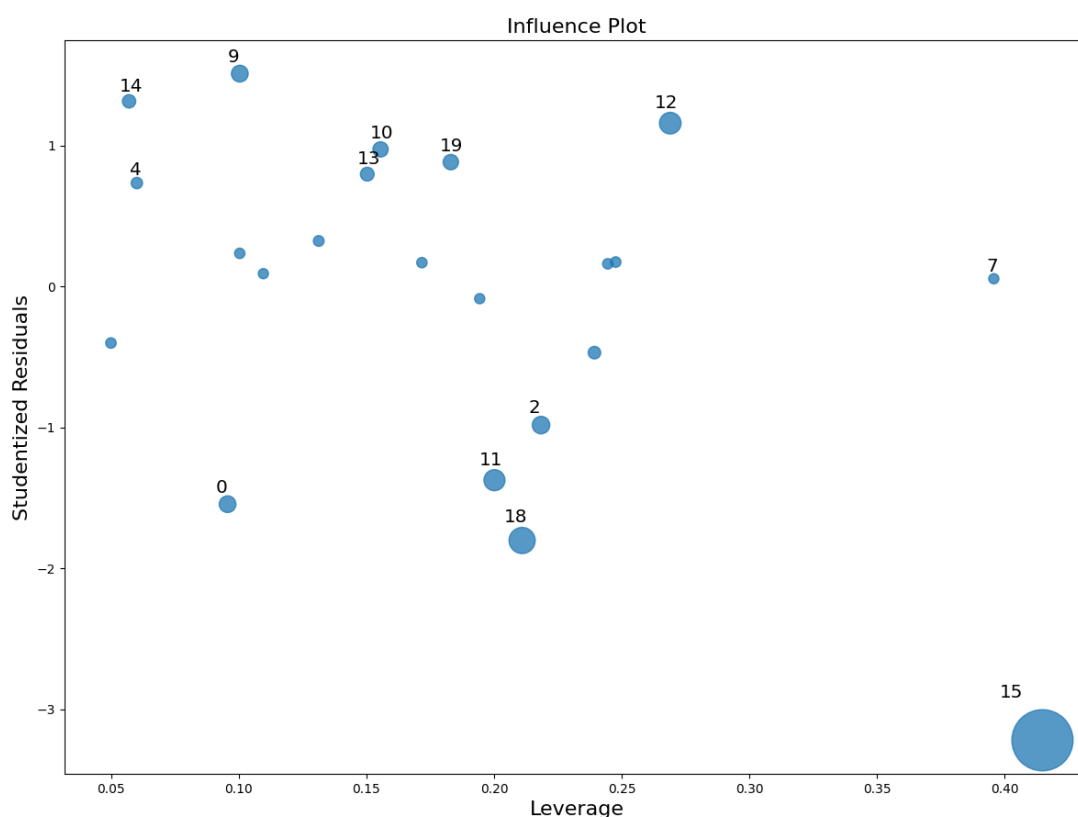


In Figures 5 and 6, we can see the regression lines don't change much with or without the extreme point. However, in Figure 7, the extreme point does significantly alter the regression result. An extreme point that can significantly alter the regression result is called an **influential point**. We need to identify influential points in the dataset and study why they are extreme (a special event or erroneous data input) because they can potentially alter the regression result. Then, we will run one model with the influential point and one without the influential point. Afterwards, we compare the model results and evaluate which results fit our modeling goal the best.

We use **Cook's distance (Cook's D)** to identify influential points in the dataset. *Cook's D* is a metric to calculate the prediction difference from the model with a data point and without a data point. The higher the *Cook's D*, the greater the influence the data point has on estimating the regression model. Oftentimes, a data point with a *Cook's D* larger than 1 is considered an influential point.

Figure 8: Influence Plot for KO's Excess Return Dataset

```
[14]: # Influence plot
fig = sm.graphics.influence_plot(result, criterion="cooks", alpha=0.5)
fig.tight_layout(pad=1.0)
```



```
[15]: # Finding most influential points
influence = result.get_influence()
inf_sum = influence.summary_frame().sort_values("cooks_d", ascending=False)
inf_sum.head(30)
```

```
[15]:
```

	dfb_Intercept	dfb_Dow_Q_EX_R	dfb_Pepsi_Q_EX_R	dfb_REV_Q_R	cooks_d \
15	-1.889683	2.264630	-0.430822	0.727270	1.208639
18	-0.124451	-0.824470	0.459042	-0.147797	0.193197
12	0.291357	0.075669	-0.235436	0.625953	0.121111
11	-0.148011	0.132958	-0.454322	-0.227980	0.112746
2	-0.159230	-0.367882	0.329489	0.120792	0.067781
9	0.443542	-0.212676	-0.033271	-0.237637	0.059381
0	-0.255128	0.021979	-0.109979	-0.299454	0.058620
19	0.043818	0.341470	-0.123705	0.131239	0.044112
10	0.279508	-0.201537	0.101374	-0.316001	0.043645
13	0.179775	-0.221049	0.250137	-0.160919	0.028646
14	0.238351	-0.011381	0.063579	-0.137167	0.025077
6	-0.036470	-0.064849	-0.081808	0.179166	0.018177
4	0.126545	0.044447	-0.028209	0.087034	0.008822
20	0.051847	-0.036797	0.069661	0.050285	0.004132
17	0.046248	0.017238	-0.048866	0.083383	0.002594
21	-0.080076	0.021967	-0.023327	0.017680	0.002226
8	0.019401	-0.023776	0.059551	0.036331	0.002196
1	0.067437	-0.038266	0.004845	-0.040646	0.001618
16	0.023219	0.019100	0.007730	-0.055804	0.001555
7	0.028748	-0.003076	-0.031566	0.008274	0.000511
5	-0.018065	-0.028030	0.033928	0.001168	0.000485
3	0.012976	-0.008273	0.022625	-0.012147	0.000267

	standard_resid	hat_diag	dffits_internal	student_resid	dffits
15	-2.611742	0.414779	-2.198763	-3.220750	-2.711472
18	-1.699993	0.210985	-0.879084	-1.803180	-0.932443
12	1.147373	0.269001	0.696022	1.158204	0.702592
11	-1.342453	0.200156	-0.671553	-1.375293	-0.687981
2	-0.985067	0.218387	-0.520695	-0.984210	-0.520242
9	1.458782	0.100410	0.487366	1.509731	0.504388
0	-1.489205	0.095619	-0.484230	-1.545590	-0.502564
19	0.887378	0.183060	0.420059	0.881882	0.417457
10	0.973571	0.155539	0.417827	0.972081	0.417188
13	0.804747	0.150331	0.338500	0.796534	0.335045
14	1.287766	0.057037	0.316715	1.313450	0.323032
6	-0.480737	0.239315	-0.269644	-0.470221	-0.263745
4	0.742975	0.060086	0.187853	0.733375	0.185426
20	0.330638	0.131321	0.128555	0.322303	0.125314
17	0.177546	0.247641	0.101861	0.172695	0.099078
21	-0.411542	0.049939	-0.094354	-0.401842	-0.092130
8	0.164745	0.244543	0.093731	0.160224	0.091159

1	0.240863	0.100382	0.080458	0.234455	0.078317
16	0.173185	0.171725	0.078857	0.168446	0.076699
7	0.055859	0.395732	0.045204	0.054289	0.043934
5	-0.089631	0.194377	-0.044026	-0.087125	-0.042796
3	0.093098	0.109637	0.032669	0.090496	0.031756

Figure 8 is an **influence plot**. It helps us visualize the *Cook's D* among all data points. On the horizontal axis of the plot is hat value. Hat value measures the difference between the exogenous values of a data point and the mean of exogenous values in the dataset. On the vertical axis of the plot is studentized residuals. These are the standardized residuals from the predicted endogenous value of the data point from the model. The bubbles in the plot indicate the size of the *Cook's D* for each point. The bigger the bubble, the higher the *Cook's D* for a data point.

From the influence plot, we can see that Point 15 and Point 18 have the highest *Cook's D* values. Among the points, Point 15 has a *Cook's D* of 1.21, so we consider it an influential point.

1.7 7. Conclusion

In this lesson, we have introduced several important concepts for regression analysis. We started with the idea of an endogenous variable and an exogenous variable. An endogenous variable in a regression model is a variable whose value is determined by the model. An exogenous variable is a variable whose value is given outside of the model. Then, we discussed how a scatter plot can help us check if two variables are linearly correlated, if they are independent, or if they have extreme data points. We used the OLS method to estimate a regression model for our *KO* excess stock price return project. We learned how to use the p-value to check the significance of the coefficient estimate for each exogenous variable. We learned how to use R^2 and an adjusted R^2 to assess the overall performance of regression model results. We introduced the concept of outliers, leverage points, and influence points. We also learned how to use Cook's distance and influence plots to identify an influence point.

During this lesson, we also touched upon issues such as variable transformation and correlated exogenous variables in a model. In the upcoming lessons, we will address these issues in more depth.

All rights reserved WQU WorldQuant University QQQQ

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Orloff, Jeremy, and Jonathan Bloom. "7b: Covariance and Correlation." Introduction to Probability and Statistics, Spring 2014, Massachusetts Institute of Technology. Lecture. https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/resources/mit18_05_s22_class07-prep-b_pdf/

- **Required pages:** Entire lecture
- **Estimated reading time:** 2 hours

Lesson 2 - Correlation and Multicollinearity

December 8, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 2

1 CORRELATION AND MULTICOLLINEARITY

Reading Time	60 minutes
Prior Knowledge	Linear Algebra, Linear Regression
Keywords	Covariance and Correlation, Positively correlated variables, Negative correlated variables, Uncorrelated variables, Collinearity & Multicollinearity, Correlation Matrix, Variance Inflation Factor (VIF)

In the last lesson, we learned the basic steps to use linear regression models to analyze KO's stock excess return. We first explained the types of variables in the model and the tools to check variable correlation. We then built a linear regression model and tested the fit of the model by using various metrics and plots.

In this lesson, we are going to explore the relationship among variables in more detail. Specifically, we would like to investigate what would happen to a regression model if the model's exogenous variables are highly correlated. We will propose some methods to reduce the impact of correlated exogenous variables. We will continue to use KO's stock excess return project as our example.

```
[1]: # Libraries
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

plt.rcParams["figure.figsize"] = (12, 9)

[2]: # Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

1.1 1. Expanding Independent Variables to Study Correlation

We are going to add more exogenous variables (independent variables) into our *KO* stock excess return model. We selected five stocks to add to the regression model in order to improve our model performance. The five stocks are:

- Google
- Bank of America
- Walmart
- General Electric
- Starbucks

We're following the last lesson's method to get the quarterly excess returns for the five stocks and then add them to the regression model. Hence, we are expanding our model to include eight variables. Here is the list of the variables:

- Dependent Variable: *KO* quarterly excess return (*KO_Q_EX_R*)
- Independent Variable: *Dow 30* Index quarterly excess return (*Dow_Q_EX_R*)
- Pepsi quarterly excess return (*Pepsi_Q_EX_R*)
- *KO* quarterly revenue return (*REV_Q_R*)
- Google quarterly excess return (*GOOG_Q_R*)
- Bank of America quarterly excess return (*BAC_Q_R*)
- Walmart quarterly excess return (*WMT_Q_R*)
- General Electric quarterly excess return (*GE_Q_R*)
- Starbucks quarterly excess return (*SBUX_Q_R*)

1.2 2. Covariance and Correlation

In finance, we are particularly interested in understanding how two variables interact with each other. One popular question is the following: How do two stock returns move together? An answer to this question is provided by the covariance and correlation between two stock returns. **Covariance** is a metric to measure the amount of movement the two variables exhibit. Here is the covariance formula:

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])]$$

- If the covariance has a positive sign, it means the two variables move in the same direction.
- If the covariance has a negative sign, the two variables move in opposite directions.
- If the covariance is 0, the two variables are **linearly uncorrelated (uncorrelated)**.

The higher the absolute value of the covariance of the two variables, the stronger the (positive or negative) relationship the two variables have. The downside of covariance is its value changes when the scales of two variables change. Because of this issue, we use correlation more often in data analysis.

Correlation is also a metric to measure the co-movement of the two variables. However, it eliminates the scale issue mentioned above by dividing covariance with the square root of the multiplication of the two variables' variances. Here is the correlation math formula:

$$\text{Corr}(X, Y) = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}$$

- Unlike covariance, the value of correlation is limited between -1 and 1 .
- If the correlation of two variables is greater than 0 , the two variables are **positively correlated**.
- If the correlation of two variables is less than 0 , the two variables are **negatively correlated**.
- If two variables are **perfectly positively correlated**, the correlation will be 1 .
- If two variables are **perfectly negative correlated**, the correlation will be -1 .
- If the correlation is 0 , the two variables are **linearly uncorrelated**.

In Figure 1, we illustrate the relationship of two variables based on their correlation in graphs.

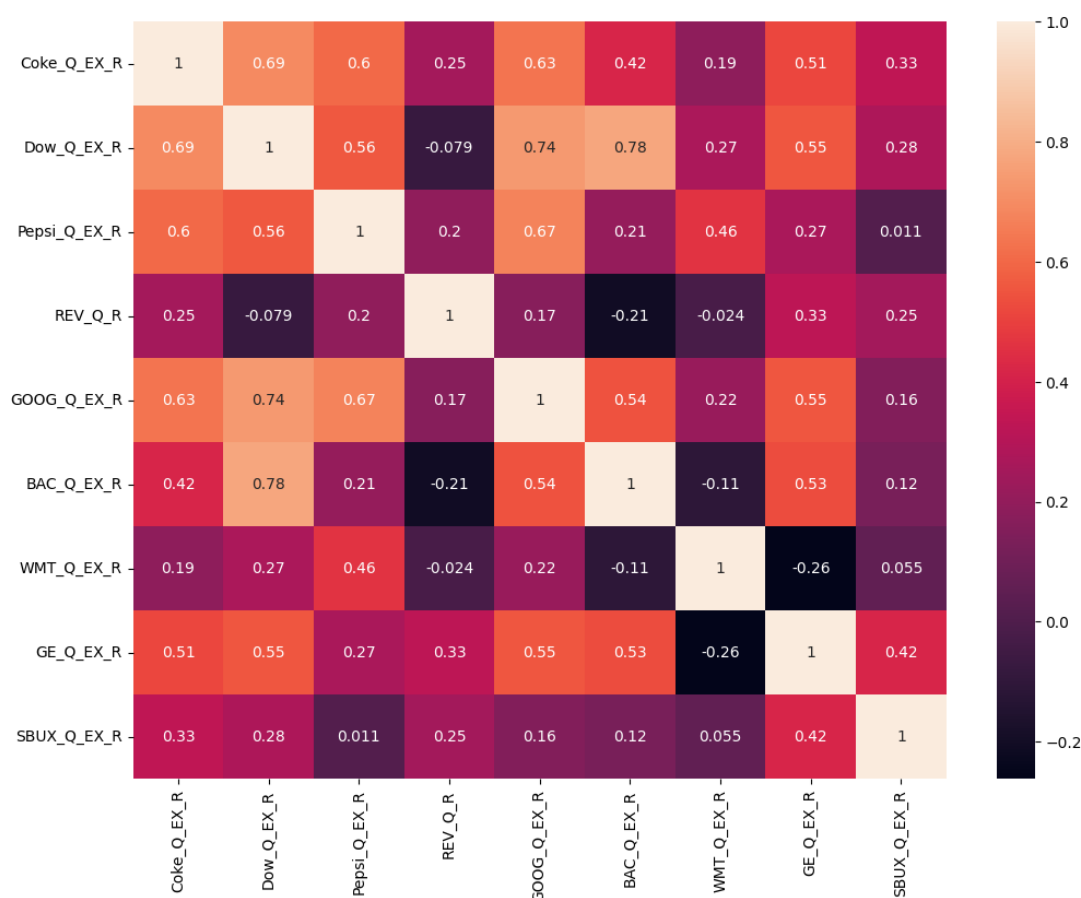
Figure 1: Correlation Demonstration Graphs

1.3 3. Collinear Variables

Most of the financial data we use is collected from various sources and is out of our control in terms of how it is generated. Because we cannot control how financial data is generated, oftentimes, many financial variables move together in systematic ways. Such a problem is called **collinearity** for two variables. We also call the two variables **correlated variables**. If there are more than two variables that are correlated, it is called **multicollinearity**. How can we detect multicollinearity?

A **correlation matrix** is the best way to investigate multicollinearity. A correlation matrix is an ensemble of the correlations of all variables. Let's look at our KO stock excess return model to understand the impact of multicollinearity. Figure 2 is the correlation matrix for the dependent variable and independent variables in our model. The correlation of different variables is shown on the upper right triangle of the correlation matrix. The diagonal of the correlation matrix is the correlation for the variable itself, which is always 1 . The lower left triangle of the correlation matrix actually provides the same correlation information as the upper right triangle.

```
[3]: # Correlation plot of all the variables
data = data_set[
    [
        "Coke_Q_EX_R",
        "Dow_Q_EX_R",
        "Pepsi_Q_EX_R",
        "REV_Q_R",
        "GOOG_Q_EX_R",
        "BAC_Q_EX_R",
        "WMT_Q_EX_R",
        "GE_Q_EX_R",
        "SBUX_Q_EX_R",
    ]
]
c = data.corr()
sns.heatmap(c, annot=True)
plt.show()
```



Now let's look at the numbers in the correlation matrix.

1.3.1 3.1 Dependent Variable and Independent Variables

As seen in the matrix, let's examine our dependent variable. *KO*'s excess return is fairly positively correlated with three independent variables: *Dow 30* Index excess return, Pepsi excess return, and Google excess return. Therefore, we can see that these three variables are pretty good at explaining *KO*'s excess return variation.

1.3.2 3.2 Low-Correlated Independent Variables

Now let's turn our focus just on independent variables. First, we'll first look at *KO*'s revenue growth, Walmart excess return and Starbucks' excess return. They are the 3 variables that have low correlation with other independent variables. However, they also have low correlation with our dependent variable: *KO* excess return.

1.3.3 3.3 High-Correlated Independent Variables

In the matrix, the *Dow 30* Index excess return has a high correlation with Google's excess return and Bank of America's excess return. Google's excess return also has a high correlation with

Pepsi's excess return. With these four fairly correlated independent variables in our model, will they present any issues to our regression result?

1.4 4. Impact of Multicollinearity in Regression Model

1. If independent variables are highly correlated, the variances, standard errors and covariances of the coefficients from the regression model may be large. Then, the estimates of the coefficient will be less precise because the confidence intervals for the coefficient estimates will be wide.
2. The high standard errors for the coefficient estimates also mean the estimates are less likely to be statistically significant. The R^2 and adjusted R^2 may be high because the whole model is still good. The collinearity issue makes it hard to separate the individual impact from correlated independent variables in the model.
3. Even though it is hard to isolate each variable's impact in the model, the model can still be a good choice to forecast if the new data from the independent variables have the same collinear issue as the sample data used to build the model.

Let's run our expanded model and see how the multicollinearity issue influences the model result.

Figure 3: KO Excess Return Regression Model with Eight Independent Variables

```
[4]: # KO Excess Return Regression Model with Eight Independent Variables
model_1 = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + Pepsi_Q_EX_R + REV_Q_R + GOOG_Q_EX_R +
    BAC_Q_EX_R + WMT_Q_EX_R + GE_Q_EX_R + SBUX_Q_EX_R",
    data=data_set,
).fit()
model_1.summary()
```

[4]:

Dep. Variable:	Coke_Q_EX_R	R-squared:	0.633
Model:	OLS	Adj. R-squared:	0.407
Method:	Least Squares	F-statistic:	2.798
Date:	Mon, 24 Nov 2025	Prob (F-statistic):	0.0482
Time:	16:05:34	Log-Likelihood:	39.396
No. Observations:	22	AIC:	-60.79
Df Residuals:	13	BIC:	-50.97
Df Model:	8		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.0059	0.017	-0.337	0.742	-0.043	0.032
Dow_Q_EX_R	0.7091	0.471	1.504	0.156	-0.309	1.727
Pepsi_Q_EX_R	0.2686	0.267	1.005	0.333	-0.309	0.846
REV_Q_R	0.1007	0.121	0.834	0.419	-0.160	0.362
GOOG_Q_EX_R	0.0233	0.208	0.112	0.912	-0.425	0.472
BAC_Q_EX_R	-0.0762	0.156	-0.487	0.634	-0.414	0.262
WMT_Q_EX_R	-0.1313	0.213	-0.617	0.548	-0.591	0.328
GE_Q_EX_R	-0.0040	0.106	-0.037	0.971	-0.234	0.226
SBUX_Q_EX_R	0.0824	0.135	0.608	0.554	-0.210	0.375

Omnibus:	1.980	Durbin-Watson:	2.255
Prob(Omnibus):	0.372	Jarque-Bera (JB):	1.638
Skew:	-0.628	Prob(JB):	0.441
Kurtosis:	2.543	Cond. No.	45.6

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[5]: # Parameters with high precision
model_1.summary2().tables[1]
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Intercept	-0.005859	0.017409	-0.336544	0.741832	-0.043469	0.031751
Dow_Q_EX_R	0.709085	0.471371	1.504303	0.156406	-0.309250	1.727419
Pepsi_Q_EX_R	0.268625	0.267324	1.004868	0.333299	-0.308893	0.846144
REV_Q_R	0.100740	0.120734	0.834399	0.419126	-0.160090	0.361570
GOOG_Q_EX_R	0.023328	0.207589	0.112377	0.912241	-0.425141	0.471798
BAC_Q_EX_R	-0.076156	0.156360	-0.487054	0.634329	-0.413950	0.261639
WMT_Q_EX_R	-0.131313	0.212743	-0.617238	0.547737	-0.590916	0.328290
GE_Q_EX_R	-0.003975	0.106396	-0.037365	0.970762	-0.233830	0.225879
SBUX_Q_EX_R	0.082359	0.135423	0.608162	0.553557	-0.210205	0.374923

In the above results, we can see no independent variable estimates are significant (p -values are all larger than 0.05). Even the *Dow 30* Index excess return is not significant in this model. This variable was significant in the model in Lesson 1. The R^2 is higher in this model (0.633) compared to the R^2 in Lesson 1's model (0.604). This is expected since we have added more independent variables to this model. However, the adjusted R^2 in this model (0.407) is significantly lower than the adjusted R^2 in the last model (0.538).

1.5 5. Methods to Address Multicollinearity

Method 1:

This first method to reduce the impact of multicollinearity is to drop independent variables that have high correlation. Usually, the correlation between 0.8 and 0.9 is considered high correlation. You can drop one of the correlated variables from the model and run it again to see if the model improves. The problem with using this method is that it only considers the relationship between two variables one at a time. This method doesn't address multicollinearity with more than two variables. Sometimes, using correlation alone cannot find this issue.

Method 2:

The second method is to treat one of the independent variables in the model as a dependent variable and run a regression with the rest of the independent variables. We can use this method to determine if any of the independent variables can be explained by the other independent variables. Let's use *KO's* excess return model as an example.

Say we make *Dow 30* Index excess return as a dependent variable and run a regression model with the rest of the independent variables. All rights reserved WQU WorldQuant University QQQQ

Figure 4: Regression Model to Check Multicollinearity Among Independent Variables

```
[6]: # Regression model to check multicollinearity among independent variables
model_dow = smf.ols(
    "Dow_Q_EX_R ~ Pepsi_Q_EX_R + REV_Q_R + GOOG_Q_EX_R + BAC_Q_EX_R +
    WMT_Q_EX_R + GE_Q_EX_R + SBUX_Q_EX_R",
    data=data_set,
).fit()
model_dow.summary()
```

Dep. Variable:	Dow_Q_EX_R	R-squared:	0.856
Model:	OLS	Adj. R-squared:	0.784
Method:	Least Squares	F-statistic:	11.91
Date:	Mon, 24 Nov 2025	Prob (F-statistic):	6.13e-05
Time:	16:05:34	Log-Likelihood:	51.064
No. Observations:	22	AIC:	-86.13
Df Residuals:	14	BIC:	-77.40
Df Model:	7		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.0061	0.010	-0.625	0.542	-0.027	0.015
Pepsi_Q_EX_R	0.1892	0.143	1.324	0.207	-0.117	0.496
REV_Q_R	-0.0605	0.067	-0.910	0.378	-0.203	0.082
GOOG_Q_EX_R	0.0977	0.115	0.851	0.409	-0.148	0.344
BAC_Q_EX_R	0.2319	0.063	3.658	0.003	0.096	0.368
WMT_Q_EX_R	0.1898	0.109	1.734	0.105	-0.045	0.425
GE_Q_EX_R	0.0570	0.058	0.977	0.345	-0.068	0.182
SBUX_Q_EX_R	0.0800	0.074	1.084	0.297	-0.078	0.238

Omnibus:	0.530	Durbin-Watson:	2.372
Prob(Omnibus):	0.767	Jarque-Bera (JB):	0.588
Skew:	-0.043	Prob(JB):	0.745
Kurtosis:	2.204	Cond. No.	26.6

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
[7]: # Parameters with 6 significant digits
model_1.summary2().tables[1]
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Intercept	-0.005859	0.017409	-0.336544	0.741832	-0.043469	0.031751
Dow_Q_EX_R	0.709085	0.471371	1.504303	0.156406	-0.309250	1.727419
Pepsi_Q_EX_R	0.268625	0.267324	1.004868	0.333299	-0.308893	0.846144
REV_Q_R	0.100740	0.120734	0.834399	0.419126	-0.160090	0.361570
GOOG_Q_EX_R	0.023328	0.207589	0.112377	0.912241	-0.425141	0.471798
BAC_Q_EX_R	-0.076156	0.156360	-0.487054	0.634329	-0.413950	0.261639
WMT_Q_EX_R	-0.131313	0.212743	-0.617238	0.547737	-0.590916	0.328290
GE_Q_EX_R	-0.003975	0.106396	-0.037365	0.970762	-0.233830	0.225879
SBUX_Q_EX_R	0.082359	0.135423	0.608162	0.553557	-0.210205	0.374923

Figure 4 shows the regression result. We can see from Figure 4 that R^2 and adjusted R^2 are high. It means that the information of Dow 30 Index excess return can be mostly explained by the rest of the independent variables. In statistics, the **variance inflation factor (VIF)** is used to determine if an independent variable has a multicollinearity issue. The VIF formula for each independent variable is as follows:

$$\text{VIF} = \frac{1}{1 - R^2}$$

Any software will calculate the VIF for each independent variable. The value of the VIF is from 1 and upward without limit.

- When R^2 is 0, the VIF is 1. It means there is no multicollinearity for this independent variable.
- The higher the VIF, the more multicollinearity is present for this independent variable.
- An independent variable with a VIF between 1 and 5 has no severe multicollinearity.
- An independent variable with a VIF that is higher than 5 indicates severe multicollinearity.

For an independent variable that has a high VIF, we can test the model with and without this variable and then check the model result.

Let's show the list of the VIF values of all independent variables.

```
[8]: labels = [
    "Intercept",
    "Dow_Q_EX_R",
    "Pepsi_Q_EX_R",
    "REV_Q_R",
    "GOOG_Q_EX_R",
    "BAC_Q_EX_R",
    "WMT_Q_EX_R",
    "GE_Q_EX_R",
    "SBUX_Q_EX_R",
]

variables = model_1.model.exog
vif = [variance_inflation_factor(variables, i) for i in range(variables.
    ↪shape[1])]

pd.DataFrame(vif[1:], labels[1:], columns=["VIF"])
```

```
[8]:
```

	VIF
Dow_Q_EX_R	6.957440
Pepsi_Q_EX_R	2.748897
REV_Q_R	1.612413
GOOG_Q_EX_R	3.241519
BAC_Q_EX_R	4.269485
WMT_Q_EX_R	2.184550
GE_Q_EX_R	3.096814

SBUX_Q_EX_R 1.511017

We can see *Dow 30* Index has a VIF value larger than 5. Since this variable has a severe multicollinearity issue, we need to dig further into this variable and maybe drop it from the model.

Method 3:

The third method we are going to discuss is principal component analysis. Principal component analysis can not only help to mitigate the issue of multicollinearity in a regression but is also an important analytical tool for other financial topics. Because principal component analysis has widespread application in finance, we will spend the next lesson discussing this topic.

1.6 6. Conclusion

In this lesson, we introduced covariance and correlation to measure the relationship of two variables. Then, we explained the meaning of collinearity and multicollinearity and how to use a correlation matrix to identify their existence among independent variables in a regression model. The existence of multicollinearity will make coefficient estimates less precise and not statistically significant. We proposed three methods to handle multicollinearity: First, check correlations between each of the independent variables and drop one of the independent variables if the correlation is higher than 0.8. Second, use the VIF metric to test independent variables with a VIF greater than 5. Third, use information from a principal component analysis.

In the next lesson, we will continue our exploration of how to mitigate multicollinearity. Specifically, we will introduce principal component analysis to tackle this issue.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Grimson, Eric et al. "Monte Carlo Simulation." Introduction to Computational Thinking and Data Science, Fall 2016, Massachusetts Institute of Technology. Lecture. <https://ocw.mit.edu/courses/6-0002-introduction-to-computational-thinking-and-data-science-fall-2016/resources/lecture-6-monte-carlo-simulation/>

- **Required pages:** Entire video
- **Estimated reading time:** 1 hour

Lesson 3 - Principal Component Analysis and Variable Transformation

December 8, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 3

1 PRINCIPAL COMPONENT ANALYSIS AND VARIABLE TRANSFORMATION

Reading Time	60 minutes
Prior Knowledge	Linear Regression, Correlation, Linear Algebra
Keywords	Principal Component Analysis, Dimension Reduction, Standardized Variable, Covariance Matrix, Eigenvector & Eigenvalue, Raw Variables & Transformed Variables, Box and Cox Method

In the last lesson, we discussed the multicollinearity issue in a linear regression model. We proposed some methods to handle multicollinearity. The last one is principal component analysis. In this lesson, we will introduce principal component analysis and discuss how it can help to mitigate multicollinearity issue.

We then move to the next topic we brought up in Lesson 1: variable transformation. We will explain why we need to apply variable transformation and how to conduct transformation.

```
[1]: # Libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.formula.api as smf
from scipy.stats import boxcox
from sklearn import preprocessing
from sklearn.decomposition import PCA
```

1.1 1. Principal Component Analysis

1.1.1 1.1 Overview

Often in a project, we need to handle many variables. For example, there are 500 stocks in the *S&P 500* index. If you are tasked to study this index, it would be time consuming to investigate each stock. You need a more efficient method to study the components in this stock index. Among many variables, usually, some of them can be correlated and some of them cannot. Correlated variables can cause a multicollinearity issue, as we discussed in the last section. Is there a way to reduce the impact of multicollinearity and also reduce the amount of variables to a manageable level?

Principal Component Analysis (PCA) is a statistical technique that can help us tackle the above problems. PCA has the following properties:

1. PCA creates new variables called **principal components** from current variables in the dataset. All principal components are independent from each other.
2. Principal components will preserve as much information as possible in the current dataset. What do we mean by preserving information from a dataset? It means that the new principal components will be able to explain the variation in the dataset.
3. PCA also ranks each principal component's ability to explain the dataset variation. The first principal component has the highest ability to explain the variation of the dataset.

With the above properties from PCA, we can select the first few principal components instead of all variables for analysis. The benefit of using a smaller number of principal components to run analysis is important when there are too many variables to consider in a model. You can choose fewer variables without losing too much information and run the analysis. That's why PCA is also considered a "dimension reduction" method. The other great benefit of PCA is that all principal components are independent. Geometrically, this means that in the new coordinate system, all components are orthogonal or perpendicular to each other. Statistically, this means they are uncorrelated. When principal components are selected to run a regression model, there won't be a multicollinearity issue.

1.1.2 1.2 The Basics of Principal Components

In this section, we are going to take a deeper look at principal components. As discussed in the above section, PCA creates principal components (new variables) from original variables in a dataset. A principal component is a linear function of the original variables. For example, if we have two variables in a dataset, Var_1 and Var_2 , then PCA will transform Var_1 and Var_2 and create the following two principal components:

$$PC_1 = a_1 \text{Var}_1 + a_2 \text{Var}_2$$

$$PC_2 = b_1 \text{Var}_1 + b_2 \text{Var}_2$$

Since we only have two variables in our example, we can draw two-dimensional graphs to demonstrate the concept.

Figure 1: Principal Component Analysis Data Transformation Graph

The scatter plot on the left of Figure 1 is the dataset with original variables. Through the transformation of PCA, we can see the new data distribution under principal components on the right of Figure 5. In the left graph, the data points scatter between both Var_1 and Var_2 axes. After the PCA transformation, new data points now mostly gather round the PC_1 axis. What does this mean for data analysis? In the original dataset, you need Var_1 and Var_2 to explain the variation of the data points. In the transformed dataset from PCA, PC_1 alone can explain most of the variation in the dataset. PC_2 doesn't provide much help in explaining the variation. We can also see that PC_1 and PC_2 form a 90-degree angle. We can say that PC_1 and PC_2 are orthogonal, meaning that PC_1 and PC_2 are independent. Because PC_1 explains most of the data variation, you can use one variable instead of two for analysis. In other words, instead of analyzing the data in two dimensions, you can analyze the data in one dimension now. That's why many scientists use PCA as a dimension reduction method for their data analysis work.

1.1.3 1.3 Derivation of Principal Components

From the example in the last section, we know we need to get the transformation matrix

$$\begin{pmatrix} a_1 & a_2 \\ b_1 & b_2 \end{pmatrix}$$

in order to get principal components. In this section, we will outline the steps for how to get the transformation matrix.

1.3.1 Calculate Standardized Dataset In the above example, we have Var_1 and Var_2 in our dataset. We need to standardize Var_1 and Var_2 . Here is the formula to standardize a variable:

$$Standardized\ Variable = \frac{Original\ Variable - Mean}{Standard\ Deviation}$$

All the standardized variables will have the same scale, which means mean = 0 and variance = 1.

1.3.2 Calculate Covariance Matrix of the Standardized Variables Covariance is a metric to determine the relationship of the two variables.

$$Covariance = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{N}$$

where N is the number of data points

A positive covariance means that the two variables are moving in the same direction and vice versa.

A covariance matrix is a matrix that contains all pairwise covariances from all variables. From our two-variable example, a covariance matrix for two variables will be a 2 X 2 covariance matrix as follows.

$$\begin{pmatrix} Cov(Var_1, Var_1) & Cov(Var_2, Var_1) \\ Cov(Var_1, Var_2) & Cov(Var_2, Var_2) \end{pmatrix}$$

The diagonal elements of the covariance matrix are also the variance of the variables according to covariance's formula.

1.3.3 Calculate Eigenvectors and Eigenvalues Once we obtain the covariance matrix, we can use linear algebra to calculate eigenvectors and eigenvalues from the covariance matrix. What are eigenvectors and eigenvalues? It is best to explain them in our example.

In our two-variable example, (a_1, a_2) and (b_1, b_2) are the two column vectors in our transformation matrix

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}$$

We know (a_1, a_2) are the coefficients to transform Var_1 and Var_2 to PC_1 . (a_1, a_2) is the eigenvector for PC_1 and (b_1, b_2) is the eigenvector for PC_2 . The eigenvalue of PC_1 is the value that shows the amount of data variance PC_1 explains. The eigenvalue of PC_2 is the value that shows the amount of data variance PC_2 explains. Since PC_1 is the first principal component, its eigenvalue is the largest. The way PCA orders principal components is that the principal component with the largest eigenvalue is the first one and the one with the second largest eigenvalue will be the second. In general, most software doesn't readily provide eigenvalues. They instead provide a proportion of the total variance explained by the principal component. This proportion is calculated from the eigenvalue of the principal component.

1.3.4 Transform the Data Using PCA After getting the transformation matrix

$$\begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \end{pmatrix}$$

we can generate new variables, PC_1 and PC_2

$$PC_1 = a_1 \text{Var}_1 + a_2 \text{Var}_2$$

$$PC_2 = b_1 \text{Var}_1 + b_2 \text{Var}_2$$

1.2 2. Apply Principal Component Analysis to KO's Excess Return Project

In our *KO* excess return project, we find there is multicollinearity among independent variables. In this section, let's use PCA to see if we could reduce independent variables in the model and mitigate multicollinearity.

Figure 2: Principal Component Analysis Result Summary for 8 Independent Variables

```
[2]: # Download the dataset
data_set = pd.read_csv("fin_data_set.csv")
```

```
[3]: # Select 8 variables and standardise
pc = data_set[
    [
        "Dow_Q_EX_R",
        "Pepsi_Q_EX_R",
        "REV_Q_R",
        "GOOG_Q_EX_R",
```

```

        "BAC_Q_EX_R",
        "WMT_Q_EX_R",
        "GE_Q_EX_R",
        "SBUX_Q_EX_R",
    ]
]
pca_data = preprocessing.scale(pc)
pca = PCA(n_components=8)
pca.fit(pca_data)

```

[3]: PCA(n_components=8)

```

[4]: # Get proportions of variance and cumulative proportion of variance
pr_var = pca.explained_variance_ratio_
cum_pr = np.cumsum(pca.explained_variance_ratio_)
ind = ["Proportion of variance", "Cumulative proportion of variance"]
cols = ["PC1", "PC2", "PC3", "PC4", "PC5", "PC6", "PC7", "PC8"]
pd.DataFrame(np.vstack((pr_var, cum_pr)), ind, columns=cols)

```

```

[4]:
          PC1      PC2      PC3      PC4 \
Proportion of variance      0.417539  0.193287  0.171490  0.108867
Cumulative proportion of variance 0.417539  0.610826  0.782316  0.891183

          PC5      PC6      PC7      PC8
Proportion of variance      0.042000  0.028306  0.026893  0.011619
Cumulative proportion of variance 0.933183  0.961489  0.988381  1.000000

```

Let's look at the proportion of variance and cumulative proportion in the above chart. According to the chart, we can see that the first principal component accounts for 42% of the total variance in the data. The first six principal components explain about 96% of the total variance.

Next, we would like to see how PCA can help us reduce the number of independent variables in the model. We know each principal component is a linear function of eight original independent variables. We can choose the original variable with the highest absolute value of the coefficient in the linear combination as a proxy of that principal component.

Figure 3: Coefficients (Loadings) of 8 Principal Components

```

[5]: # Coefficients (Loadings) of 8 Principal Components
pc_res = pd.DataFrame(pca.components_.T, index=list(pc.columns), columns=cols)
pc_res

```

```

[5]:
          PC1      PC2      PC3      PC4      PC5      PC6 \
Dow_Q_EX_R  0.501013  0.081975 -0.201491  0.151476  0.172552 -0.161302
Pepsi_Q_EX_R 0.370323  0.416369  0.215351 -0.267349 -0.486188 -0.340799
REV_Q_R      0.089143 -0.205496  0.723855 -0.314653  0.525310  0.024685
GOOG_Q_EX_R  0.481635  0.115076  0.033447 -0.219037 -0.147889  0.789841
BAC_Q_EX_R   0.399072 -0.169106 -0.470595  0.014795  0.462351 -0.034007
WMT_Q_EX_R   0.116011  0.659655  0.213805  0.389464  0.370728 -0.129019

```

GE_Q_EX_R	0.399918	-0.444523	0.093024	-0.117590	-0.150220	-0.440478
SBUX_Q_EX_R	0.198186	-0.321096	0.334835	0.769979	-0.249503	0.146820

	PC7	PC8
Dow_Q_EX_R	-0.116793	0.780570
Pepsi_Q_EX_R	-0.423131	-0.200210
REV_Q_R	-0.213269	0.069342
GOOG_Q_EX_R	0.241923	-0.037978
BAC_Q_EX_R	-0.323626	-0.520391
WMT_Q_EX_R	0.392582	-0.214001
GE_Q_EX_R	0.622124	-0.127903
SBUX_Q_EX_R	-0.239618	-0.106832

Figure 3 shows the coefficients of eight independent variables for each principal component. The coefficients here can also be called loadings. For PC_1 , the Dow 30 Index excess return has the highest absolute value of the coefficient. This means that the Dow 30 Index excess return has the largest impact on PC_1 . Therefore, we can use the Dow 30 Index excess return as a proxy for PC_1 . Following the same logic, we can use Walmart's excess return as a proxy for PC_2 . We also know that the first six principal components cover around 96% of the data variation. In this case, we can just choose *Dow 30* Index excess return, Walmart excess return, *KO*'s revenue growth, Starbucks excess return, and Google excess return to run a regression model.

Let's rerun to our revised regression model with fewer independent variables to see the result.

Figure 4: KO Excess Return Revised Model Result Applying PCA Information

[6]: *# OLS for revised model*

```
model_4 = smf.ols(
    "Coke_Q_EX_R ~ Dow_Q_EX_R + REV_Q_R + GOOG_Q_EX_R + WMT_Q_EX_R + ↵
    ↵SBUX_Q_EX_R",
    data=data_set,
).fit()
model_4.summary()
```

[6]:

Dep. Variable:	Coke_Q_EX_R	R-squared:	0.584
Model:	OLS	Adj. R-squared:	0.454
Method:	Least Squares	F-statistic:	4.493
Date:	Mon, 24 Nov 2025	Prob (F-statistic):	0.00950
Time:	16:34:15	Log-Likelihood:	38.030
No. Observations:	22	AIC:	-64.06
Df Residuals:	16	BIC:	-57.51
Df Model:	5		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	-0.0109	0.014	-0.757	0.460	-0.042	0.020
Dow_Q_EX_R	0.6091	0.286	2.133	0.049	0.004	1.214
REV_Q_R	0.1418	0.103	1.380	0.186	-0.076	0.359
GOOG_Q_EX_R	0.1060	0.177	0.600	0.557	-0.268	0.480
WMT_Q_EX_R	-0.0004	0.144	-0.003	0.998	-0.305	0.304
SBUX_Q_EX_R	0.0551	0.117	0.472	0.644	-0.193	0.303
Omnibus:	1.148	Durbin-Watson:	1.998			
Prob(Omnibus):	0.563	Jarque-Bera (JB):	1.031			
Skew:	-0.466	Prob(JB):	0.597			
Kurtosis:	2.493	Cond. No.	30.4			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

From the result in Figure 4, we can see that the *Dow 30* Index has become significant, and the adjusted R^2 has improved from 0.406 at the beginning of this lesson to 0.454. By using the information provided by PCA, we can reduce the impact of multicollinearity among independent variables. All rights reserved WQU WorldQuant University QQQQ

1.2.1 2.1 Use Principal Components in a Regression Model

As you can see from the previous example, it is hard to integrate principal components directly into a regression model. If we use a principal component directly as an independent variable in a regression model, it will be hard to interpret the meaning of the coefficient estimate. Since principal components are a linear combination of several financial variables, they don't translate directly into a specific financial concept. However, we can investigate the loadings for each principal component and pick the original variable with the highest absolute value of the loadings as a proxy for that principal component for regression. This is what we did in the last example. Another way is to also look for a financial theme from the loadings of a principal component and select original variables related to the theme.

We have also noticed that by selecting the top variables from PCA for data analysis, we lose some accuracy in representing complete data variation. We mitigate the multicollinearity from the dataset at the expense of losing some ability to explain the whole data variation. How to choose the degree of the trade-off between reducing multicollinearity and losing data explaining accuracy will depend on a researcher's experience in data analysis.

1.3 3. Variable Transformation

In the previous sections, we looked at the relationships among variables and their impact on linear regression model results. In this section, we turn our focus to the individual variable itself. In the field of data analysis, oftentimes, the raw data we collect is not ready for analysis. Even if the data is in a form ready for analysis, sometimes, the variables in the dataset don't meet the model assumptions and therefore require some transformation work. In this section, we will discuss when and how to transform variables as well as choices for transforming variables and how to analyze regression models with transformed variables.

1.3.1 3.1 Raw Variables and Transformed Variables

A **raw variable** is a variable in its original form when the data is collected. The concept is different from raw data. Raw data is the dataset that comes straight from the data source. There might be missing values, outliers, wrong data input, etc. A researcher often needs to go through certain steps to clean the dataset for use. A raw variable is a variable from the cleaned dataset that has not otherwise been manipulated. For example, the X variable is from a cleaned dataset. X is a raw variable. If $Y = 2X$, then Y is not a raw variable. Y is a **transformed variable**.

1.3.2 3.2 When Do We Need to Transform Variables?

Situation 1: The raw variables are not in the proper form for analysis.

Sometimes, the data you collect is not in a suitable or practical form for your analysis. For example, in our KO stock excess return model, the raw variables we obtained from the internet are KO 's stock price, Google's stock price, etc. However, their stock price levels are different. Once we transform stock prices to price returns, the two variables are now on the same scale and easier to compare and analyze.

Sometimes, there are not many raw variables in a dataset. As a data scientist, you can try to create new variables by manipulating raw variables. For example, suppose you are analyzing consumer activities in a supermarket. You have each consumer's sales numbers and the number of trips made in a year. From the two raw variables, you can create a new variable called *Sales Per Trip*, which is sales number divided by number of trips. This new variable might provide new insight for the analysis. Alternatively, perhaps you might form a variable that looks at related purchases, such as always buying two items together. This new variable might provide different insight for the analysis.

Situation 2: The raw variables don't meet analysis assumptions.

In financial models, usually there are assumptions about the variables in the model. For example, in a regression model, the key assumption is that the dependent variable and independent variables have a linear relationship. For a simple regression model, it is easy to draw a scatter plot for the dependent variable and independent variable to see if they form a linear relationship in the plot.

Figure 5: Non-Linear Relationship between X and Y

Figure 5 is a scatter plot for two variables, X and Y . We can see X and Y don't have a linear relationship in the plot.

Another assumption in many financial models to consider is whether a variable is normally distributed. Suppose one of the assumptions for a financial model is to required variable Y to be normally distributed. We can draw a histogram for the variable and see if the histogram shows a bell-shaped pattern. Figure 6 shows three histograms for three different Y distributions.

Figure 6: Histograms with Different Distributions

1.3.3 3.3 Methods to Transform Variables

Method 1: Transformation for Linear Relationship

If one assumption of a model requires variables to be linearly dependent but this relationship doesn't exist, we need to transform the variables so that the linear relationship is present. The methods to try are log transformation, square root transformation, or cube root transformation. These methods have two things in common:

- They preserve the order of the data points
- They “tame” outliers by making them less extreme. Note the change in the scales between the two graphs.

Following up from the example in Figure 5, we can do a log transformation on X . In Figure 7, we can see the original scatter plot from Figure 5 on the left and the transformed X in the scatter plot on the right. You can see that there is a linear relationship between Y and $\ln(X)$.

Figure 7: Pre-Transformation Scatter Plot vs. Post-Transformation Scatter Plot

Sometimes, you may need to transform both x and y to obtain the linear relationship for the two variables. Variable transformation is a trial and error process. Based on your model assumptions, you will need to try to either transform one variable or two or even more to achieve your modeling goal.

Method 2: Transformation for Normal Distribution

When a model requires a variable to be normally distributed but the variable does not meet the assumption, we need to transform this variable to be normally distributed. We can use log transformation, square root transformation, or cube root transformation to get the result. One more thing to consider is the outliers for a non-normal variable with skewed distribution. If you drop the outliers on the tail, it may bring the variable back to normal distribution.

Another method to consider is the **Box and Cox method**. Box and Cox method is a variable transformation method that can transform a non-normally distributed variable to a normally distributed variable. Assume we have a variable Y which is not normally distributed. The Box and Cox method will try to find a λ satisfying the following rules and converting Y to a normally distributed variable.

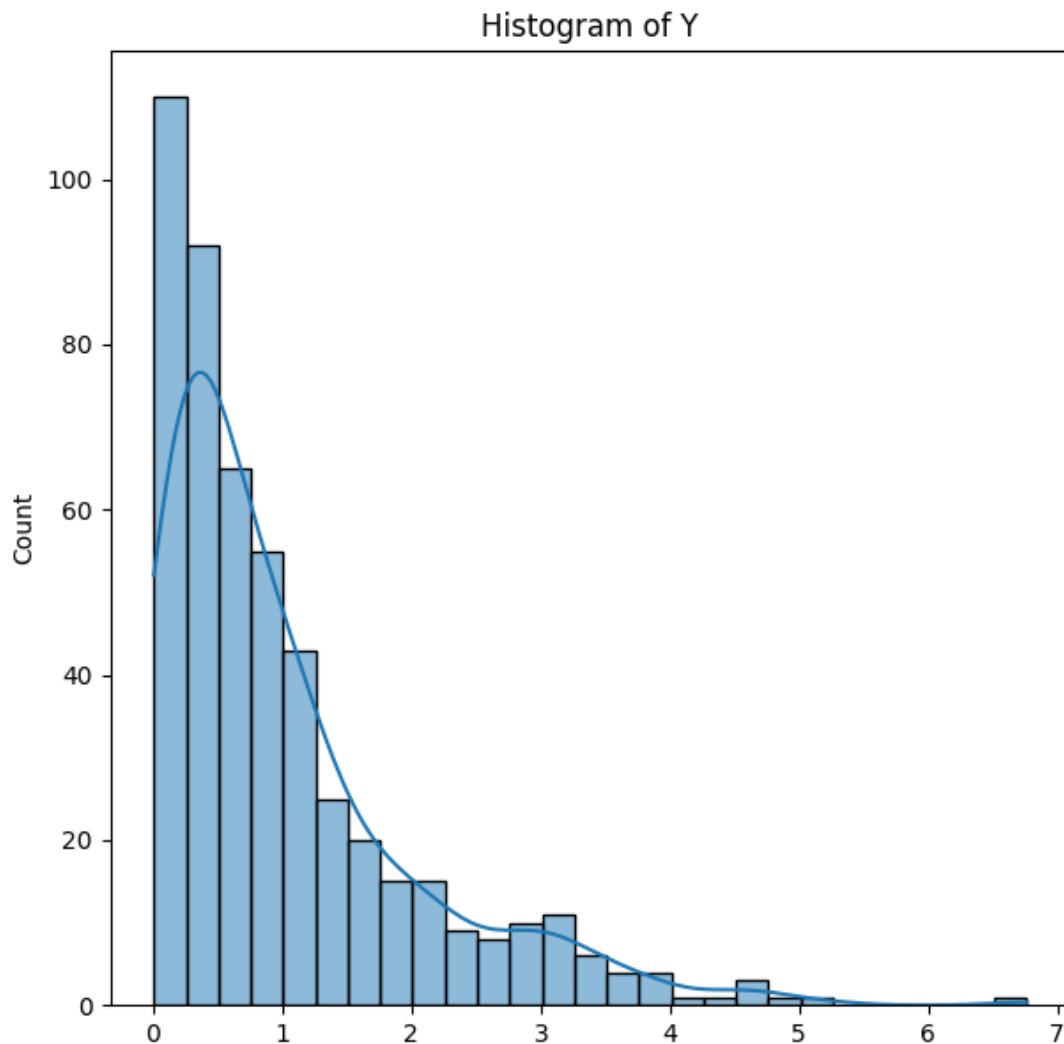
$$\text{new}Y = \begin{cases} \frac{(Y^\lambda - 1)}{\lambda}, & \text{if } \lambda \neq 0 \\ \log(Y), & \text{if } \lambda = 0 \end{cases}$$

Box and Cox Method Transformation

```
[7]: # Generate random dataset
np.random.seed(0) # to make reproducible example
Y = np.random.exponential(size=500)

# Plot distribution of data
plt.figure(figsize=(7, 7))
```

```
plt.title("Histogram of Y")
sns.histplot(Y, kde=True)
plt.show()
```

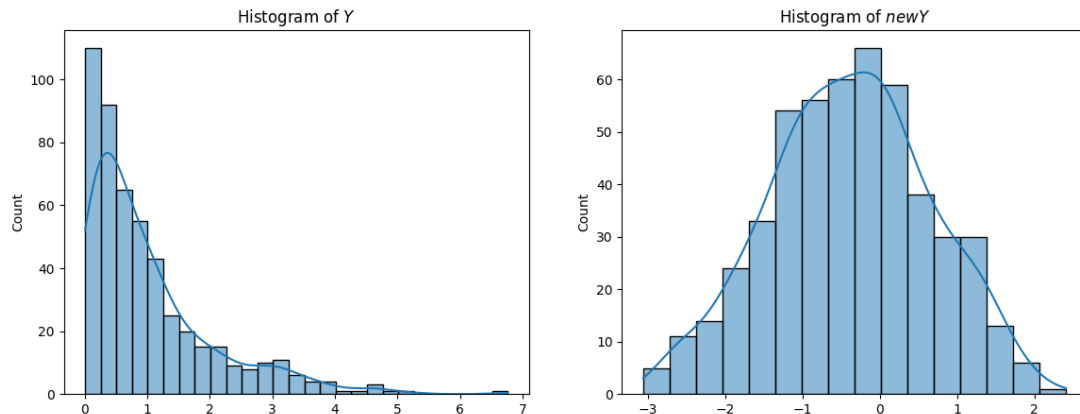


```
[8]: # Apply Box-Cox transformation on original data
new_Y, best_lambda = boxcox(Y)
print("Optimal Box-Cox transformation lambda:", best_lambda)

# Plot distributions of original data
plt.figure(figsize=(14, 5))
ax1 = plt.subplot(121)
sns.histplot(Y, kde=True, ax=ax1)
plt.title("Histogram of $Y$")
```

```
# Plot distribution of transformed data
ax2 = plt.subplot(122)
sns.histplot(new_Y, kde=True, ax=ax2)
plt.title("Histogram of $newY$")
plt.show()
```

Optimal Box-Cox transformation lambda: 0.2325441269447074



The figure above shows us the histogram of our original variable Y on the left and the histogram of the transformed variable Y using the Box and Cox method on the right. We can see from the figure that the original variable Y is right skewed. With Box and Cox transformation, the new variable is normally distributed.

1.3.4 3.4 Concluding Words for Variable Transformation

Variable transformation is both an art and a science. Oftentimes, we need to draw a scatter plot and/or histogram to check the variables. Then, we apply different transformations to the variable. We use the transformed variable in the regression model and check the coefficient significance and model fit. If one transformation doesn't work, we try another transformation. It's an iterative process until the regression model result is good.

1.4 4. Conclusion

In this lesson, we first introduced principal component analysis. We explained what principal components are and demonstrated the derivation of principal components. We then learned how to use the information obtained from principal component analysis to rerun a regression model to mitigate the multicollinearity issue. The second topic we discussed in this lesson is variable transformation. We showed that when the dependent variable and independent variables are not linearly related, we need to transform independent variables to make sure a linear relationship exists to run a regression model. We also showed what to do when we need to convert a non-normally distributed variable to a normally distributed one.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Fee, Michale, and Daniel Zysman. "Principal Components Analysis." Introduction to Neural Computation, Spring 2018, Massachusetts Institute of Technology. Lecture. <https://ocw.mit.edu/courses/9-40-introduction-to-neural-computation-spring-2018/resources/17/>

- **Required pages:** Entire video
- **Estimated reading time:** 1 hour and 15 minutes

2. Osborne, Jason. "Improving Your Data Transformations: Applying the Box-Cox Transformation." *Practical Assessment, Research, and Evaluation*, vol. 15, no. 12, 2010, pp. 1–9.
https://www.researchgate.net/publication/284261483_Improving_your_data_transformations_Applying_Box-Cox_transformations_as_a_best_practice

- **Required pages:** Entire article
- **Estimated reading time:** 1 hour

Lesson 4 - Monte Carlo Simulation

December 8, 2025

0.1 FINANCIAL ECONOMETRICS

MODULE 1 | LESSON 4

1 MONTE CARLO SIMULATION

Reading Time	60 minutes
Prior Knowledge	Time series analysis, Basic statistics
Keywords	Deterministic Model, Stochastic Model, Monte Carlo Simulation, Law of Large Numbers, Central Limit Theorem

In the previous lessons, we learned that a linear regression model always includes an error term. This error term provides a random characteristic to the prediction from the linear regression model. In this lesson, we will explore this random characteristic more. Specifically, we will discuss deterministic models and stochastic models. Then, we will introduce Monte Carlo simulation.

```
[1]: # Load libraries
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams["figure.figsize"] = (12, 9) # Figure size and width
```

1.1 1. Deterministic Models

Deterministic models are models that do not have a random term. Let's look at one example. Assume you put \$10 dollars in an investment account at the beginning of each month. The account would give you a **fixed annual interest** of 40% compounding monthly. We can use the following equation to calculate the amount of money you will get by the end of a year.

$$\sum_{i=0}^{11} P_t \left(1 + \frac{r}{12}\right)^{12-i}$$

where P_t is the amount of money put in at the beginning of each month and r is the fixed annual interest rate.

Figure 1: Deterministic Model: Cumulative Account Balance with Monthly \$10 Dollars Investment and Fixed Annual Interest

```
[2]: # Deterministic Model
month = np.linspace(start=1, stop=12, num=12)
x = []

for i in range(12):
    x.append(10 * (1 + 0.4 / 12) ** (12 - i))

cum_x = np.cumsum(x)

# Plot
plt.plot(cum_x, marker="o")
plt.title("Deterministic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(
    round(cum_x[11], 2), (10.5, cum_x[11] - 10), fontsize=13
) # Annotate last point
plt.show()
```

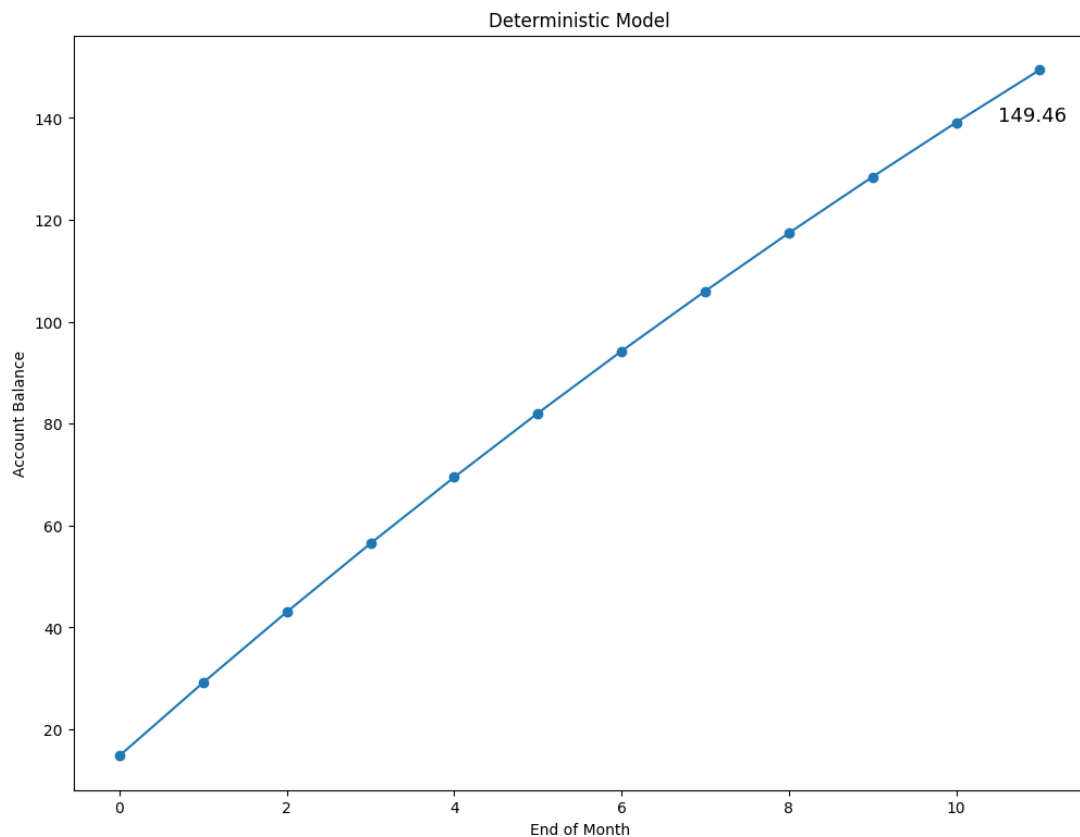


Figure 1 shows the cumulative account balance at the end of each month. With this model, we can accurately predict how much money is in the account at each month's end. There is no uncertainty within this model. If one invests \\$10 dollars at the beginning of every month, one will get \\$149.46 dollars by the end of the year. One can be 100% sure how much money will be in the account after one year if the amount of monthly investment is known in advance.

A deterministic model will produce the same result given the input information is fixed. Repeated prediction processes with the same input for a deterministic model will not change the prediction outcome. Other common deterministic models are principal component analysis and factor analysis. The classification or cluster analysis we will discuss later in Lesson 4 is also a deterministic model. In contrast to the deterministic model, a stochastic model will have a random component in the model.

1.2 2. Stochastic Models

A **stochastic model** is a model that has a random and unknown component in it. Let's go back to the example in the last section. Assume you put \$10 dollars in an investment account at the beginning of each month. However, now the account would give you a **variable annual interest** adjusting and compounding monthly. The annual interest rate is a random variable and, for instance, follows a normal distribution with mean 40%. We cannot know the definitive numbers of all future interest revenues from the investment at time 0. There is uncertainty in predicting the future cumulative account balance. We can use the following equation to calculate the amount of money you will get by the end of a year.

$$\sum_{i=0}^{11} P_t \left(1 + \frac{r_t}{12}\right)^{12-i}$$

where P_t is the amount of money put in at the beginning of each month and r_t is the annual interest rate adjusted each month.

In this case, we have introduced an uncertain component, the variable annual interest rate. In the following figure, we sample 12 interest rates from the normal distribution with mean 40% and apply them to the above model to come up with one possible account balance outcome for each month in a year.

Figure 2: Stochastic Model: Cumulative Account Balance with a Monthly \$10 Dollar Investment and Variable Annual Interest All rights reserved WQU WorldQuant University QQQQ

```
[3]: # Stochastic model 1
np.random.seed(123)
int_1 = np.random.normal(0.4, 0.7, 12) # np.random.normal(mean, sd, size)
bal_1 = []

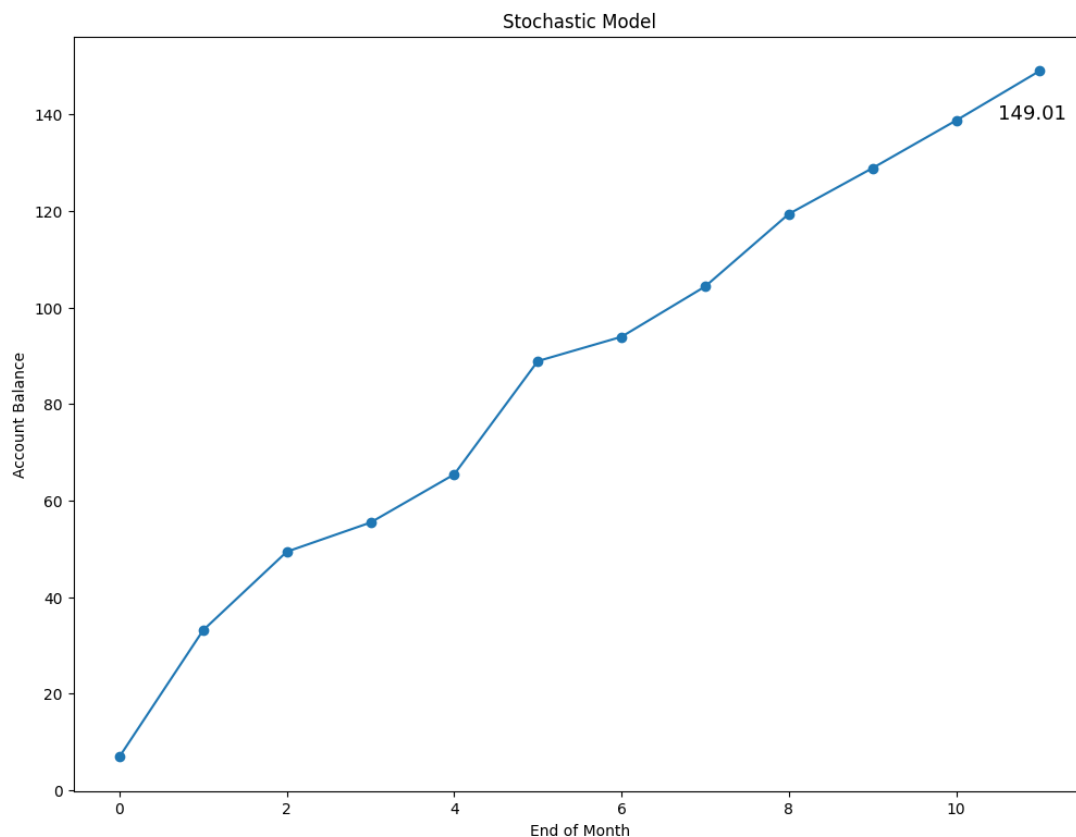
for i in range(12):
    bal_1.append(10 * (1 + int_1[i] / 12) ** (12 - i))
```

```

cum_bal_1 = np.cumsum(bal_1)

# Plot
plt.plot(cum_bal_1, marker="o")
plt.title("Stochastic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(round(cum_bal_1[11], 2), (10.5, cum_bal_1[11] - 10), fontsize=13)
plt.show()

```



From figure 2, we can see the trend in the stochastic model is going up and is similar to the case for the deterministic model. However, a stochastic model has a squiggly line instead of a smooth line as in a deterministic model. This squiggly pattern comes from the fact that the variable annual interest rates applied to each month are different. Unlike a deterministic model, at time 0, the stochastic model cannot predict a precise number for what the account balance will be by the end of the year. We know the variable annual interest comes from a normal distribution with a mean of 40%, but we do not know the definite numbers. What we can do is sample 12 numbers from the distribution and apply them to the model to predict the account balance in a year. This is what we did in this example. However, if we repeat the same prediction process—sample another 12 numbers from the distribution and apply them to the model—we will very likely get another

predicted account balance in a year. Every time we repeat the prediction process, we get a different account balance. The key difference between the deterministic model and stochastic model is that the prediction from the deterministic model is always the same no matter how many times one repeats the prediction process. However, the prediction outcome will be different from a stochastic model each time a researcher repeats the prediction process as shown in the following plot.

Figure 3: Three Different Prediction Paths for a Stochastic Investment Account Balance Model

```
[4]: # Three different prediction paths

# Stochastic model 2
np.random.seed(375)
int_2 = np.random.normal(0.4, 0.7, 12)
bal_2 = []

for i in range(12):
    bal_2.append(10 * (1 + int_2[i] / 12) ** (12 - i))

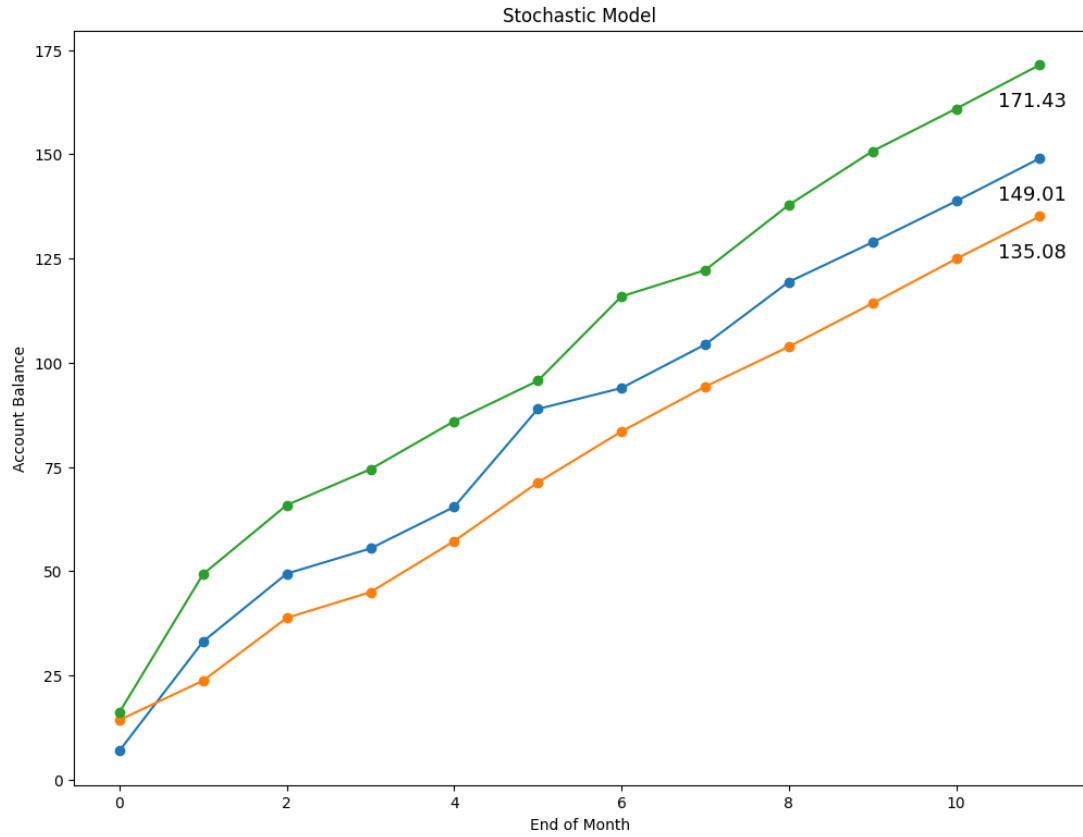
cum_bal_2 = np.cumsum(bal_2)

# Stochastic model 3
np.random.seed(999)
int_3 = np.random.normal(0.4, 0.7, 12)
bal_3 = []

for i in range(12):
    bal_3.append(10 * (1 + int_3[i] / 12) ** (12 - i))

cum_bal_3 = np.cumsum(bal_3)

# Plot three stochastic model lines
plt.plot(cum_bal_1, marker="o")
plt.plot(cum_bal_2, marker="o")
plt.plot(cum_bal_3, marker="o")
plt.title("Stochastic Model")
plt.xlabel("End of Month")
plt.ylabel("Account Balance")
plt.annotate(round(cum_bal_1[11], 2), (10.5, cum_bal_1[11] - 10), fontsize=13)
plt.annotate(round(cum_bal_2[11], 2), (10.5, cum_bal_2[11] - 10), fontsize=13)
plt.annotate(round(cum_bal_3[11], 2), (10.5, cum_bal_3[11] - 10), fontsize=13)
plt.show()
```



In figure 3, we show three predicted cumulative account balance paths of the stochastic model. As you can see, all three of them follow the same trend in general with different variations. Their predicted outcomes for cumulative account balances by the end of a year are also different, as the plot indicates. This is the key difference between a deterministic model and a stochastic model. In a stochastic model, for each prediction activity, because of the random component in the model, you get a new line when you have a new set of sampled input data. In a deterministic model, no matter how many times you run prediction activities, you should always get the same line.

The linear regression model is an example of stochastic models. As long as there is a random element with a probability distribution in a model, this model is a stochastic model.

1.3 3. Monte Carlo Simulation

1.3.1 3.1 An Overview of Monte Carlo Simulation

In the last section, we learned that a stochastic model would generate a different prediction outcome for each prediction run due to the random component within the model. The prediction outcome of a stochastic model is not definite. We can generate samples from independent random variables in a stochastic model and apply them to the model to derive one predicted outcome. In the last section, we drew three sets of sampled data to generate three predicted lines in figure3. We can repeat the process multiple times, and we will get multiple predicted cumulative account balances. These predicted outcomes will form a distribution. We can calculate the average and use the average as

our final prediction for the cumulative account balance. However, how accurate is this averaged cumulative account balance? We can use the variance derived from the distribution from sampled data to conduct statistical tests to check the accuracy of the averaged outcome. This technique is called Monte Carlo (MC) simulation.

In general, we only have one set of realized datasets in a real-world project. For example, we only have one set of Google's historical stock price to analyze. However, if we have some idea what the probability distribution of Google's stock price should be or use Google's historical price to form an empirical distribution, with MC simulation, we can sample repeatedly from this distribution to generate several sets of samples and apply them to the model to generate predicted outcomes. Then, we use these predicted outcomes to conduct inferential statistics. Hence, we can see MC simulation as a random sampling technique from a probability distribution to estimate an unknown population parameter.

MC simulation is based on two important statistics theories: law of large numbers (LLN) and central limit theorem (CLT).

1.3.2 3.2 Law of Large Numbers and Central Limit Theorem

Let's quickly review LLN and CLT first. Assume we have a sequence of identically and independently distributed n random variables $x_1, x_2, \dots, x_n$. All the random variables have mean μ and variance σ^2 . Let's define the average of the sequence as follows:

$$y_n = \frac{x_1 + x_2 + \dots + x_n}{n}, \text{ where } n = 1, 2, \dots$$

The law of large numbers states that y_n will converge to μ as $n \rightarrow \infty$.

Let's also define z_n as follows:

$$z_n = \frac{(x_1 - \mu) + (x_2 - \mu) + \dots + (x_n - \mu)}{\sqrt{n}}$$

The central limit theorem states that z_n will converge to $N(0, \sigma^2)$ as $n \rightarrow \infty$.

1.3.3 3.3 Estimators for Unknown Stochastic Model Outcome Expected Value and Variance

According to the law of large numbers, the average of a very large sample is a good approximation of the unknown model outcome, meaning μ . Hence, if we run random samplings for m times to get m model outcomes and m is large enough, the sample average

$$\bar{\mu} = \frac{1}{m} \sum_{i=1}^m x_i$$

will be a good approximation of μ .

Also, we can obtain an estimate of the variance from the samples using the formula:

$$\sigma_m^2 = \frac{\sum_{i=1}^m (x_i - \bar{\mu})^2}{m-1}$$

According to central limit theorem, we can write the following equation:

$$\bar{\mu} - \mu \sim \frac{\sigma}{\sqrt{m}}z, \text{ where } z \sim N(0, 1)$$

Hence, $\bar{\mu}$ converges to μ with speed of $\frac{\sigma}{\sqrt{m}}$ as m is large enough.

With the above information, we can construct a 95% confidence interval as follows:

$$\left[\bar{\mu} - \frac{1.96\sigma_m}{\sqrt{m}}, \bar{\mu} + \frac{1.96\sigma_m}{\sqrt{m}} \right]$$

where σ_m is the standard deviation of sampled outcomes.

From introductory statistics, we know that the true value has a 95% chance of being within the confidence interval. The width of the confidence interval is used as a measure of the precision of the estimate.

We can see that the width is controlled by two elements: the number of sampling frequencies and the variance of the sampled outcomes.

- a.** The size of number of sampling frequencies m : as m goes up, the width of the CI shrinks.
- b.** The variance of the sampled outcomes σ_m^2 : as σ_m^2 goes up, the width of the CI expands.

Therefore, in order to increase the precision of the Monte Carlo simulation, we can either increase the number of simulations or reduce the variance of the outcome variable. In this lesson, we will focus on increasing the number of simulations to improve Monte Carlo simulation results. For those who are interested in reducing the variance of the outcome variable, you can research variance reduction techniques online.

1.3.4 3.4 Monte Carlo Simulation Application: Investment Account Balance Stochastic Model

Let's use our investment account balance stochastic model as an example to run Monte Carlo simulation. The stochastic model is as follows:

$$\sum_{i=0}^{11} 10 \left(1 + \frac{r_t}{12} \right)^{12-i}$$

Where r_t is the annual interest rate adjusted each month.

We will put in \$10 dollars at the beginning of each month for a year or 12 months. The variable annual interest rate follows a normal distribution with mean of 40% and standard deviation of 0.7. We would like to forecast the account balance by the end of a year.

Here are the steps to run a Monte Carlo simulation to get samples of year-end account balance:

1. Sample 12 rates from variable annual interest rate distribution $N(40\%, 0.7)$.
2. Apply 12 rates into the model and generate one data point for year-end account balance.
3. Repeat step 1 and step 2 m times to get m data points for year-end account balance.
4. Calculate the mean and standard deviation of the m data points and construct confidence intervals.

For our investment account balance model, we will run MC simulations 100 times, 1,000 times, 100,000 times, and 1 million times to compare the results. The following plots and tables show the simulation results from different runs.

Figure 4: MC Simulation with 100 Trials, 1,000 Trials, 100,000 Trials and 1 Million Trials

```
[5]: # Monte Carlo Simulation

# Helper function that plots histogram and prints statistics
def Hist(trials):
    res = []

    # Simulate Monte Carlo sample
    for j in range(trials):
        int = np.random.normal(0.4, 0.7, 12)
        bal = []

        for i in range(12):
            bal.append(10 * (1 + int[i] / 12) ** (12 - i))

        res.append(sum(bal))

    # Plot Histogram
    plt.hist(res, bins=30, density=True, histtype="stepfilled", alpha=0.7)
    plt.title("Histogram of {} Simulations".format(trials))
    plt.xlabel("Cumulative Account Balance")
    plt.ylabel("Frequency")

    # Add statistics
    avg = np.mean(res) # Mean
    sigma = np.std(res) # Std
    lowb = avg - 1.96 * sigma / np.sqrt(trials) # Low Bound
    upb = avg + 1.96 * sigma / np.sqrt(trials) # Upper Bound
    width = upb - lowb

    textstr = "{0} trials\n Average: {1:.2f}\n Standard Dev: {2:.2f}\n\n Upper Bound: {3:.2f}\n Low Bound: {4:.2f}\n Width: {5:.2f}".format(
        (trials), (avg), (sigma), (upb), (lowb), (width))
```

```
)  
return textstr
```

```
[6]: # Plot four histograms  
plt.figure(figsize=(16, 12))  
  
# No. trials = 100  
ax1 = plt.subplot(221)  
fig1 = Hist(100)  
plt.text(  
    0.95,  
    0.85,  
    fig1,  
    ha="right",  
    va="center",  
    fontsize=10,  
    bbox={"facecolor": "orange", "alpha": 0.5},  
    transform=ax1.transAxes,  
)  
  
# No. trials = 1,000  
ax2 = plt.subplot(222)  
fig2 = Hist(1000)  
plt.text(  
    0.95,  
    0.85,  
    fig2,  
    ha="right",  
    va="center",  
    fontsize=10,  
    bbox={"facecolor": "orange", "alpha": 0.5},  
    transform=ax2.transAxes,  
)  
  
# No. trials = 100,000  
ax3 = plt.subplot(223)  
fig3 = Hist(100000)  
plt.text(  
    0.95,  
    0.85,  
    fig3,  
    ha="right",  
    va="center",  
    fontsize=10,  
    bbox={"facecolor": "orange", "alpha": 0.5},  
    transform=ax3.transAxes,  
)
```

```

# No. trials = 1,000,000
ax4 = plt.subplot(224)
fig4 = Hist(1000000)
plt.text(
    0.95,
    0.85,
    fig4,
    ha="right",
    va="center",
    fontsize=10,
    bbox={"facecolor": "orange", "alpha": 0.5},
    transform=ax4.transAxes,
)
plt.show()

```

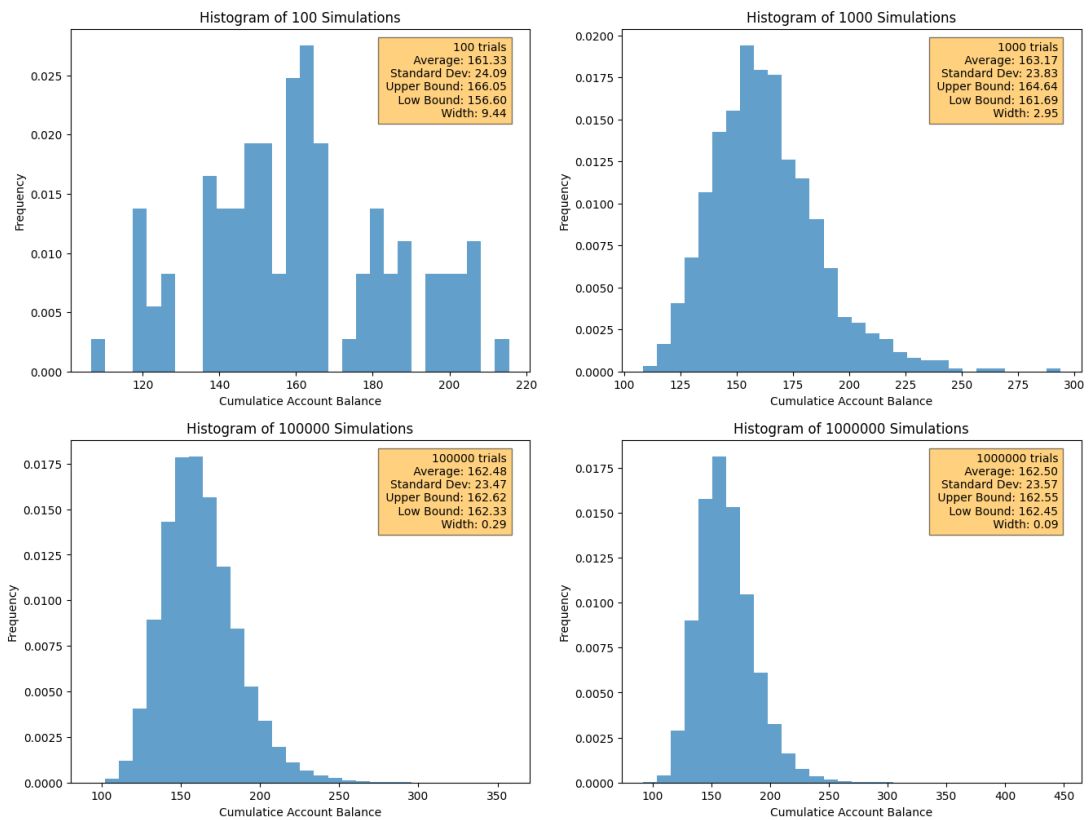


Figure 4 shows the histograms of predicted account balance distribution with a different number of trials. We can see as the number of trials increases, the shape of the histogram will become more concentrated.

In figure 4, we can see as the number of trails increases, the average predicted year-end account

balance converges to 162.5. We can see when the number of trials increases to 1,000 runs from 100 runs, the average drops a lot and then stays very stable around 162.5. Also, the width of the confidence intervals drops a lot from 1,000 runs to 100 runs. From these two numbers, we can say 1,000 trials are enough to get convergent results for a year-end account balance estimator.

1.4 4. Conclusion

In this lesson, we introduced some new concepts. We first discussed deterministic models, that is, models without any random component or uncertainty within the model. When one uses a deterministic model for prediction, the prediction outcome is certain. Then, we considered stochastic models, i.e., models with a random component. Because of the randomness, if one runs simulations many times and uses simulation results, the predictions obtained will in general be different for different simulations. In order to analyze multiple prediction outputs, we introduced the concept of Monte Carlo simulation. A Monte Carlo simulation repeatedly samples data from the model to generate predicted outcomes and perform inference about the model.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.